Recursive Self Improvement for Embodied Agents

A Survey of Mechanisms Evaluation and Open Challenges

Yu Huang[*], Yue Chen[&], Zijiang Yang[#]
[*]: Institute of Advanced Technology, University of Science and Technology of China, China
[#]: School of Computer Science & Technology, University of Science and Technology of China, China
[&]: Futurewei Technology Inc., San Jose, USA

Abstract

Embodied foundation models have expanded the range of instructions, observations, and actions that robots can process. Autonomy in execution, however, does not automatically provide autonomy in improvement. Robots must still diagnose failures, acquire useful experience, select interventions, and determine whether accepted changes improve future performance. Recursive self-improvement (RSI) connects these activities through persistent feedback and inheritance, with the stronger objective of improving the mechanisms that produce subsequent improvements. This survey distinguishes within-episode correction, persistent self-improvement, successive version inheritance, and meta-level improvement. It connects large-model learning, sequential agency, and RSI theory, and compares general agent harnesses with their embodied counterparts. It develops a four-axis framework covering improvement targets, update mechanisms, autonomy over improvement decisions, and evidence conditions. Using this framework, we examine reflection and knowledge revision, executable-program evolution, adaptive experience acquisition, parameter learning, world-model revision, and cross-layer intervention selection. We compare representative systems while retaining their assumptions about feedback, validation, transfer, and cost. We also review benchmarks and associated datasets for manipulation, long-horizon decision making, memory, and world modeling, separating pretraining data, improvement experience, candidate validation, and final evaluation. Existing results support several forms of component-level and system-level improvement, but capability gains, improvement autonomy, and demonstrated recursive gains require different evidence. We identify failure attribution, reliable feedback, physical reset, competence preservation, and independent evaluation of revised improvers as central research priorities.

Keywords: embodied agents; recursive self-improvement; agent harnesses; vision-language-action models; world models; evaluation

1 Introduction

Embodied intelligence requires an agent to change its environment through action and to use the resulting observations in subsequent decisions. Foundation models enable broader instruction understanding, semantic transfer, and the composition of existing capabilities. Vision-language-action models (VLAs) connect visual and linguistic inputs to robot actions, while world models and world action models (WAMs) provide predictive representations for planning and learning. These advances expand available capabilities, but do not determine how a deployed system should use or improve them [38,51,96,117].

A failed task can reflect an incorrect object reference, missing evidence, inappropriate skill selection, an unsuitable starting pose, an inadequate motor policy, or an inaccurate dynamics model. Increasing model capacity may not repair a poor invocation, and repeatedly rewriting instructions cannot guarantee acquisition of a missing motion. Recent systems address these problems through persistent memory, code-based skills, execution harnesses, autonomous practice, and environment modeling [112,139,140,144,147,149,150,152–154,157,159,163,164]. Their shared challenge is to decide what should change after a failure and which evidence justifies retaining that change.

An agent harness organizes the observations, tools, state, execution, verification, and recovery surrounding a model. The embodied-harness perspective emphasizes runtime responsibilities and distinguishes operational execution from infrastructure supporting data collection, training, and evaluation [142]. RSI extends the unit of analysis from an execution to a retained system update. Its stronger recursive objective concerns the procedure that produces future updates, rather than only the policy or skill library produced by that procedure [133,160].

We therefore do not treat every feedback-driven robot as an RSI system, or use model-weight updates as the sole boundary. A frozen backbone can support persistent improvement through tools, skills, and executable workflows.

Conversely, reinforcement learning can improve a policy while leaving its objective, sampling strategy, training algorithm, and acceptance rule fixed. Capability growth, responsibility for improvement decisions, and the effectiveness of a revised improver must be assessed separately.

The survey first develops foundations from large-model learning, agent theory, and RSI, then compares general and embodied harness methods. It makes four contributions. First, it separates improvement targets, mechanisms, autonomy, and evidence in a common analytical framework. Second, it connects different intervention surfaces through failure attribution. Third, it examines benchmarks, datasets, and protocol differences that constrain numerical comparisons. Finally, it develops a research agenda around physical costs, safe inheritance, and controlled meta-improvement evaluation. The review is a focused narrative synthesis of primary papers and official resources consulted through 20 October 2026. Reported results are retained within their original experimental settings; we do not pool them into a meta-analysis or independently reproduce the systems.

Related surveys establish the surrounding fields but address different review objects. Liu et al. [97] map embodied perception, interaction, agents, and sim-to-real adaptation; Feng et al. [113] connect language-based cognition with predictive world modeling. Surveys of general self-evolving agents [111,130] explain mutable components and update signals, whereas the present review asks how physical evidence supports retained updates and improved future update production. This distinction keeps enabling architectures, ordinary adaptation, and demonstrated recursion analytically separate.

Figure 1 distinguishes three coupled processes: execution, persistent system revision, and revision of the improvement procedure. The diagram is an analytical synthesis of the responsibilities discussed in this survey, rather than the architecture of a single reviewed method. Solid arrows carry execution and improvement evidence; dashed arrows show accepted changes. Physical reset remains explicit, and the execution system S and improvement procedure Φ may change on different schedules.

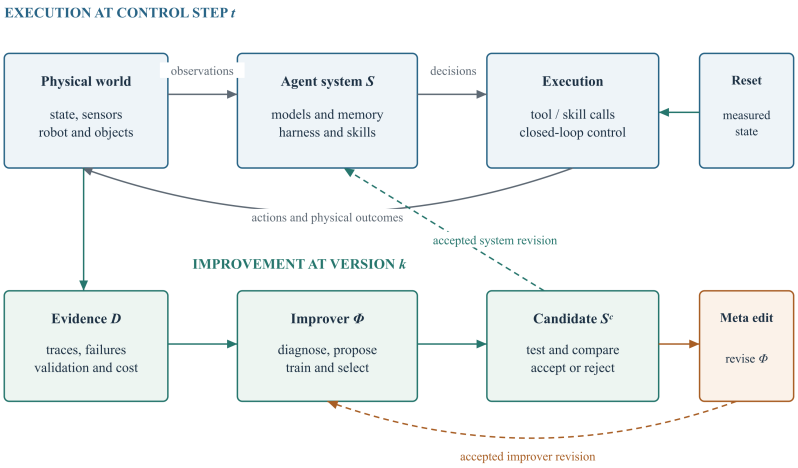


Figure 1 Execution and improvement loops

2 Theoretical Foundations

This section develops the conceptual foundations needed to analyze embodied RSI. Large-model research explains how general representations and conditional behavior are learned; agent theory explains how those capabilities participate in sequential interaction; RSI theory concerns retained changes to the agent and, in stronger forms, to its improvement procedure. These are complementary foundations. None alone establishes that repeated robot updates will be reliable or accelerate future learning.

The foundations proceed from large-model learning and reasoning to robotic model interfaces, agent memory and skills, and recursive improvement. This ordering separates inference procedures, persistent parameter learning, and changes to the procedure that produces future improvements. Sections 2.5 and 2.6 then connect these foundations to evidence-guided robot optimization.

2.1 Large Model Foundations

2.1.1 Pretraining Adaptation and Multimodal Representation

Foundation models are trained on broad data and adapted to downstream uses [30]. Zhao et al. review language-model pretraining, adaptation, utilization, and evaluation, while Li et al. trace the transition from specialist multimodal models to general assistants [59,76]. For language models, the Transformer supplies attention-based computation over sequences [9]. A common pretraining objective minimizes negative log likelihood, $\mathcal{L}(\theta) = -\mathbb{E}[\sum_i \log p_\theta(x_i|\text{prefix}_i)]$, where prefix_i contains the tokens before token i. Learning this conditional distribution can support generation, contextual inference, and code production. The objective remains a prediction objective: successful next-token prediction does not by itself specify a robot objective, a physically valid action, or a justified system update.

Scaling studies characterize empirical relationships among model size, data, computation, and language-model loss [22,37]. These results motivate allocating resources across parameters and training tokens, but their fitted regimes and objectives matter. They do not directly imply a scaling law for physical task success or recursive gain. Robot performance additionally depends on action representations, contact dynamics, observation quality, embodiment, and the distribution of interaction experience. A survey should distinguish these empirical regularities from guarantees of general intelligence or autonomous improvement.

Three adaptation pathways are especially relevant. In-context learning conditions behavior on examples or instructions without updating model parameters [24]. Parameter adaptation changes the learned mapping through additional training. External adaptation changes retrieved knowledge, executable skills, context construction, or orchestration. In-context changes may disappear when the context is discarded, whereas externally stored lessons can persist without weight updates. The location of a change therefore does not establish its lifetime, and a frozen foundation model does not imply a fixed agent.

Visual-language foundations differ in the interfaces they provide. CLIP learns transferable image–text representations; Flamingo supports interleaved visual and textual conditioning; BLIP-2 connects pretrained image encoders and language models; and visual instruction tuning supports multimodal instruction following [26,39,52,61]. For an embodied improver, these interfaces can support semantic retrieval, observation interpretation, and diagnostic dialogue. None directly substitutes for a calibrated physical state estimator or a motor controller.

2.1.2 Chain of Thought and Tree of Thoughts

Chain-of-thought (CoT) prompting conditions a language model on intermediate reasoning examples so that it generates a sequence of intermediate steps before an answer [33]. Writing the prompt as x, the intermediate text as z, and the answer as y, the joint generation can be factorized as $p_\theta(z, y|\, x) = p_\theta(z|\, x)p_\theta(y|\, x, z)$. This is a description of generated text, not proof that z faithfully exposes all internal computation. CoT prompting changes the conditioning and generation procedure; it does not itself update model weights.

Self-consistency samples several reasoning paths and aggregates their final answers, rather than committing to one greedy path [35]. Tree of Thoughts (ToT) instead represents partial solutions as thought states, generates alternatives, evaluates them, and searches with lookahead or backtracking [63]. The distinction is sequential elaboration, aggregation across sampled paths, and explicit search over partial solutions. ToT is an inference framework, not a new backbone architecture, and it is not synonymous with Monte Carlo tree search.

For embodied agents, these ideas motivate task decomposition, alternative failure hypotheses, and comparison of intervention plans. This is an analytical transfer of mechanisms, not a robotic result established by the original language benchmarks. A textual search branch is also different from a physical trial: imagined branches depend on model assumptions, whereas real trials incur action, sensing, and reset costs. Section 2.5 connects reasoning proposals to externally measured evidence and persistent memory.

Zhang et al. [83] explicitly connect CoT reasoning with language agents. This supports placing CoT and ToT in the large-model reasoning foundations, before discussing how an embodied harness grounds reasoning in actions and feedback. Li [94] further distinguishes policy, evaluator, and dynamics-model roles across tool use, planning,

and feedback learning; one language model may occupy several roles, but role assignment and shared errors must remain visible.

2.1.3 Reinforcement Learning for Fine Tuning and Inference Time Optimization

Reinforcement learning (RL) views generation as sequential decisions: the policy emits tokens or structured calls and receives feedback on outcomes or intermediate steps. Supervised fine-tuning imitates target responses; RL fine-tuning changes response probabilities to increase a reward. In RLHF, preference comparisons can train a reward model, after which policy optimization improves the language model subject to a reference-policy constraint [12,34]. Equation (1) summarizes a common KL-regularized objective, rather than the exact loss of every post-training algorithm.

$$J_{\mathrm{RL}}(\theta) = \mathbb{E}_{x\sim D, y\sim\pi_\theta(\cdot|x)}[r(x,y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{\mathrm{ref}}(y|x)}] \quad (1)$$

In Equation (1), prompts x follow a declared training distribution, y is the generated response or trajectory, r is the reward, π_{ref} is the reference policy, and β controls regularization. PPO uses a clipped policy-ratio surrogate to limit update size [12]. GRPO estimates relative advantages from groups of responses to the same problem and avoids a separately trained value critic in its original formulation [86]. DeepSeek-R1 provides an example of RL-based reasoning post-training, including verifiable outcome rewards in suitable domains [107]. Such rewards are easier to construct for checked answers or code tests than for open-ended physical success.

Outcome rewards score a completed response; process supervision supplies judgments on intermediate steps and can train a process reward model [67]. Neither guarantees physical validity when transferred to a robot. Direct Preference Optimization (DPO) is a related preference-learning alternative: it optimizes a direct preference loss without an explicit on-policy RL loop in the original algorithm [66]. DPO should therefore not be described as simply another implementation of PPO. Reward source, verifier independence, training data, and parameter scope must be disclosed.

Inference-time optimization allocates computation to the current problem through longer generation, best-of-N selection, sequential revision, or verifier-guided search [100]. The generator and verifier may have been trained with RL, but a fixed-weight search at deployment is not itself an RL weight update. Conversely, genuine test-time adaptation updates parameters online and must report the feedback used, the update scope, and whether adaptation is reset or inherited. We therefore distinguish RL-trained inference policies, search using a learned reward or value function, and online RL adaptation.

For an embodied improver, RL can train a diagnostician or experiment-selection policy, while inference-time search can compare proposed repairs before executing them. RL can also train a motor policy, which is a different intervention target. These are possible design roles, not interchangeable evidence: more sampled plans can increase current success without producing a persistent update, and a stronger RL-trained improver does not establish RSI unless its retained revisions improve subsequent improvement. Comparisons should match training resources and deployment budgets separately, including model calls, latency, physical trials, and resets.

Inference optimization also has a systems meaning. Zhou et al. [89] review data-, model-, and system-level efficiency; Jin et al. [91] examine efficient multimodal architectures; Li et al. [105] organize KV-cache management. For an embodied improver, caching and compression change latency and available evidence. They should be evaluated jointly with action deadlines, stale observations, memory loss, and downstream task outcomes, rather than treating tokens per second as an improvement metric by itself.

2.2 Robotic Foundation Models

Robotic foundation models connect multimodal representations to action and prediction. We distinguish VLA models, world models, and world action models by their modeled variables and deployed interfaces, rather than treating the names as mutually exclusive architectural categories. Table 1 summarizes their roles in improvement. Some classic world-model methods below are conceptual foundations rather than foundation models pretrained across many robots.

2.2.1 Vision Language Action Models

A vision-language-action (VLA) model learns a conditional action policy from visual observations and language instructions, optionally including proprioception and action history. A common interface is $\pi_\theta(a_{t:t+H-1} | h_t, g)$, where h_t is the observation history, g is the instruction, and H is the action-chunk horizon. Tokens, diffusion, and flow-based generation are alternative action representations; not every VLA uses next-token action prediction [69,92,96,103]. In RSI, a VLA may remain a frozen executor, receive adapter updates, or be retrained from corrective experience. These cases require different data and cost accounting.

Robot models also expose different action interfaces. RT-1 learns scalable robot control, and RT-2 connects visual-language pretraining to action prediction [48,69]. Octo and π_0 supply generalist-policy and flow-based VLA examples with adaptation interfaces [92,103]. Diffusion Policy represents action distributions through denoising, ACT predicts action chunks, PerAct uses a spatial manipulation formulation, and VIMA accepts multimodal task prompts [44,46,56,62]. Gato demonstrates a shared sequence-modeling approach across modalities and embodiments [40]. These choices determine which actions, contexts, and parameter subsets an RSI system can actually modify; they do not establish a ranking of improvement ability.

Ma et al. [93] separate VLA components, low-level action policies, and high-level task planners. We use this distinction to specify which part an RSI intervention modifies: an action decoder, the learned policy, or the surrounding planner can change independently. Huang [146] reviews robotic in-context learning with fixed deployment parameters. Context responsiveness becomes persistent improvement only when useful context-selection rules or experience survive the declared reset boundary.

2.2.2 World Models

A world model predicts consequences of candidate actions, for example $p_\psi(o_{t+1:t+H} | h_t, a_{t:t+H-1})$, or equivalent latent-state transitions and rewards. Prediction can support model-predictive planning, imagined experience, and failure diagnosis. World Models, PlaNet, Dreamer, and DreamerV3 illustrate distinct representation and control choices; learned environment models need not produce photorealistic video [14,16,21,51]. For improvement, useful prediction should preserve action-dependent consequences relevant to downstream tasks. A lower prediction loss alone does not demonstrate better control or a better improver.

World-model surveys offer complementary descriptors. Ding et al. [104] distinguish understanding current structure from predicting future dynamics. Li et al. [114] classify functionality, temporal modeling, and spatial representation; Hou et al. [121] emphasize coupling to robot learning. Mei et al. [115] examine video models as predictive infrastructure, including their physics violations. These distinctions motivate reporting representation, action conditioning, deployment role, and physical validation separately when a world model participates in self-improvement.

2.2.3 World Action Models

A world action model (WAM) couples action generation with prediction of future world evolution. DreamZero provides a concrete example that jointly models video and action using a pretrained video-diffusion backbone [117]. Equation (2) expresses this joint interface. Its factorization is an analytical description, not a requirement that every WAM implement a separately trained policy and dynamics module. The term is still used across differing implementations, so a survey should identify whether actions are jointly generated, inferred from predicted futures, or obtained through a separate decoder.

$$p_\omega(\boldsymbol{a}, \boldsymbol{o} | h_t, g) = p_\omega(\boldsymbol{a} | h_t, g) p_\omega(\boldsymbol{o} | \boldsymbol{a}, h_t, g) \quad (2)$$

In Equation (2), $\boldsymbol{a}$ and $\boldsymbol{o}$ denote future action and observation sequences over a declared horizon. The action marginal connects to a VLA-like policy interface; the conditional prediction connects to a world-model interface. Joint generation does not guarantee that the predicted scene will match the actual action outcome. WAM evaluation should therefore separate action success, action-conditioned predictive fidelity, and physical transfer, while disclosing any inverse-dynamics or decoding stage. For RSI, revising a WAM can affect both the executor and the evidence used to propose or select future updates.

WAM definitions also differ across surveys. Wang et al. [122] distinguish cascaded and joint prediction-action architectures; Shen et al. [127] include rendered, latent, and video-generation-free predictive-action methods; Lu

et al. [134] emphasize robot action interfaces and control. Accordingly, Equation (2) describes joint-generative WAMs, rather than a universal requirement for all WAMs. Cascaded or latent predictive-action systems need their own declared interfaces, and predictive coupling alone supplies no evidence of recursive gain.

Multimodal grounding remains essential across these interfaces. PaLM-E integrates embodied and visual inputs with a language model, whereas SayCan combines language relevance with affordance estimates [38,55]. A model that interprets a scene is not automatically a calibrated dynamics model, and an action generator is not automatically an independent verifier. Table 1 makes these distinctions explicit before the method review.

Table 1 Robotic model interfaces and improvement roles

Model family	Modeled interface and role	What evaluation must establish
VLA	Visual and language conditioned actions; frozen execution or policy adaptation [69,96,103].	Task success, action representation, control rate, and cost of adaptation.
World model	Future observations, latent transitions, or rewards conditional on actions; planning and imagined experience [16,21,51].	Action-conditioned prediction and downstream utility, with real-world residuals.
WAM	Coupled future actions and world evolution; joint predictive control or action extraction [117].	Physical action success, prediction consistency, and effects of decoders or transfer.

For embodied RSI, the central consequence is a division between representational capability and the evidence needed to modify behavior. An LLM may propose a useful diagnosis, a VLM may interpret an outcome, and a VLA may execute a skill, but their outputs remain conditional on training and deployment conditions. Their use as proposer, controller, and evaluator should be recorded separately. Improved explanations or model likelihood do not establish improved physical execution, and shared model priors can correlate proposal and evaluation errors.

2.3 Agent Foundations

2.3.1 Sequential Decisions and Partial Observability

An agent selects actions from its interaction history to pursue an objective. Surveys by Wang et al. and Xi et al. organize the construction, applications, and evaluation of language-model agents, with the latter describing brain, perception, and action as general architectural components [72,75]. A partially observable Markov decision process provides a decision-theoretic abstraction: latent states evolve through a transition model, observations reveal incomplete evidence, and actions affect future rewards and observations [1]. A belief state represents a distribution over possible current states. Real robot systems often approximate this information with scene representations, memory, or language summaries. These approximations may omit information required for reliable action, making state construction an intervention surface rather than a passive preprocessing step.

$$b_{t+1}(s') = \eta\, O(o_{t+1} \mid s', a_t) \sum_s T(s' \mid s, a_t) b_t(s) \tag{3}$$

Equation (3) is the standard discrete-state belief update [1]. Here t indexes control steps, s is physical state, T is the transition distribution, O is the observation likelihood, and η normalizes the belief. A robot may use approximate representations instead of an explicit belief distribution. This approximation affects which evidence a diagnosis can use.

Planning and learning occupy different roles. Planning chooses behavior using currently available knowledge; learning changes the knowledge, model, or policy used for future decisions. Replanning after a new observation can improve the current episode without creating a retained update. Updating a dynamics model, recording a reusable skill, or training a policy can affect later episodes. This distinction connects classical sequential decision making to the persistence criterion used in this survey, while allowing a single system to combine model-based planning, learned policies, and programmatic execution.

Huang [87] proposes agent levels that include memory, reflection, autonomous learning, and collaboration. This is a capability-oriented organization, not the same scale as Duan et al.'s RSI responsibility levels [133]. Guo et al. [85] survey language-model multi-agent organization, while Feng et al. [108] address embodied collaboration. For

multi-robot improvement, our extension is to record shared resources, communication delay, update ownership, and whether an apparent gain comes from coordination or from an inherited change to the improver.

2.3.2 Memory as a Persistent Information State

Memory connects current reasoning to previous experience. Working memory maintains the current task and state estimate; episodic memory retains trajectories, failures, and interventions; semantic memory stores generalized conditions and relations; procedural memory stores executable skills and invocation rules. This is a functional distinction rather than a required software partition. Generative Agents combines experience records, higher-level reflection, and retrieval for planning in a simulated social environment [60]. Voyager provides a skill-library precedent, and Reflexion provides a persistent verbal-feedback precedent [57,65]. Their different memory carriers should not be treated as equivalent evidence of physical or recursive learning.

Agent memory can be represented functionally by $m_{t+1} = U(m_t, o_t, a_t)$, with decisions conditioned on retrieved information $R(m_t, q_t)$. In a partially observable environment, memory can approximate a belief or sufficient information state, but an arbitrary summary is not guaranteed to be sufficient [1]. Within-episode state tracks the current interaction; cross-episode memory preserves selected information across resets. Storage is therefore different from learning: a retained trace becomes useful only when later retrieval and decision making use it appropriately.

Memory becomes useful for optimization when retrieval preserves the relationship between a lesson and its evidence. An entry should identify the task conditions, model and skill versions, attempted intervention, observed outcome, and uncertainty. Consolidation can turn episodes into candidate rules, but those rules need scope and retraction when later observations conflict. A memory of failed training or rejected code can prevent repeated unproductive experiments; a stale successful skill can instead produce a regression. More stored text or a larger context window does not establish better learning. Tests should compare retrieval, consolidation, retention, and incorrect reuse under matched interaction budgets.

Retrieval-augmented generation separates access to external knowledge from parameters, MemGPT organizes movement between memory levels, and Self-RAG learns retrieval and critique behavior [23,78,80]. These provide mechanisms for accessing and governing stored evidence. Their original language-task evaluations should be distinguished from retention of physical skills, calibrated state knowledge, and improvements to robotic experiment selection.

A skill library connects procedural memory to execution, whereas episodic records preserve how a skill behaved under particular conditions. The two should be linked by version and applicability, so that a successful old execution does not certify a changed implementation. Memory evaluation consequently needs read/write/consolidation ablations, cross-episode retention, retrieval errors, and interference tests, in addition to task completion. Section 2.5 explains how reflection proposes memory updates and how evidence governs their acceptance.

2.3.3 Skills Tools and Temporal Abstraction

Language-model agents add reasoning traces, tools, memory, and executable programs to this interaction loop. ReAct interleaves reasoning and actions with observations from the environment [47]. Code as Policies generates programs that compose perception and control interfaces [45]. These methods help explain how a general model can participate in an agent, but generating a plan or executable program does not alone establish that the resulting agent improves across tasks. Retention, reuse, and validation determine the self-improvement claim.

A tool is an exposed callable interface, such as a detector, planner, simulator, trainer, or robot controller. A skill is a reusable behavior that may execute multiple primitive actions through closed-loop feedback. These categories overlap: a skill can be exposed as a tool, while a training tool may change future skills without being a motor skill itself. Tool selection is therefore a decision over interfaces; skill execution additionally raises temporal abstraction and control questions [45,47,54,70].

The options framework formalizes a temporally extended skill by an initiation set, an internal policy, and a termination rule [2]. Equation (4) specifies this abstraction. A high-level agent selects an available option, the option generates lower-level actions, and control returns at termination; variable durations yield a semi-Markov

decision process. This supports analysis of skill composition without assuming that every robot implementation explicitly uses options.

$$\mathcal{O}_i = (\mathcal{I}_i, \pi_i, \beta_i), \;\; s_t \in \mathcal{I}_i, \;\; a_t \sim \pi_i(\cdot \mid h_t) \tag{4}$$

In Equation (4), $\mathcal{I}_i$ identifies admissible initiation states, π_i is the skill policy, and β_i gives termination probability or a deterministic stopping condition [2]. For a robot, an exposed skill contract should additionally specify argument grounding, required observations, actuator resources, observable postconditions, timeout, and interruption behavior. These are engineering extensions around the option abstraction, not guarantees supplied by the notation. A valid API call does not prove that a grasp or reset physically succeeded.

Embodied agency also requires temporal coordination. A high-level instruction may invoke a skill that occupies many control steps, while perception updates, interruption, and low-level control operate at different rates. A slow model call cannot replace every feedback loop. Skill entry conditions, available resources, termination tests, and interruption behavior connect deliberation to execution. The embodied-harness survey by Huang et al. organizes such responsibilities across reasoning, runtime governance, skill execution, and hardware [142]. These logical responsibilities can be distributed differently across implementations.

Skill composition must preserve compatibility between one skill's postconditions and the next skill's entry conditions. Learning a new internal policy, changing a tool wrapper, and changing the high-level invocation sequence can all improve a task through different causal routes. The harness coordinates these interfaces and resources; memory records their experience; the improver decides which artifact to revise. This separation is necessary when attributing gains to model learning, platform changes, or recursive procedure revision.

2.3.4 Execution Systems and Improvement Interfaces

For analysis, let the execution system be $S = (B, H, K, \Pi, W)$, where B denotes foundation reasoning and perception models, H the execution harness, K persistent knowledge and skill descriptions, Π executable policies or programs, and W world models. These are logical roles rather than disjoint software modules: memory and skill registries may be implemented within H. Physical state s and agent configuration S are distinct. An improvement procedure Φ uses experience D to propose, evaluate, and select changes; an evaluator E assesses the resulting system. This decomposition connects agent design to the objects and procedures that RSI can modify.

$$c_t = \text{Assemble}_{H_k}(o_{\leq t}, g, \text{Retrieve}(K_k, q_t)), \;\; a_t \sim \pi_{\theta_k}(\cdot \mid c_t) \tag{5}$$

In Equation (5), k indexes retained system versions, g is the task goal, q is a retrieval query, c is the assembled context, and θ parameterizes an executable policy. H may instead route a model output to a tool or skill. The equation abstracts this interface; it does not assume that a language model directly supplies every low-level action.

2.4 Recursive Self Improvement Foundations

RSI concerns an agent whose retained improvements can affect subsequent improvement. A useful theoretical reference is the Gödel machine, which studies self-rewrites justified by proofs relative to formal axioms and a utility specification [3]. Its conditional guarantees should not be transferred to empirical robot systems: physical environments are only partially modeled, utility specifications are incomplete, and useful proofs may be unavailable. Contemporary embodied systems predominantly use empirical search and validation. Their claims depend on experimental evidence rather than a general theorem of beneficial self-modification.

A capability update can be written as $S_{k+1} = \text{Accept}(\Phi_k(S_k, D_k))$. The procedure includes proposal, acquisition, training where applicable, candidate evaluation, and selection. A meta-level update additionally changes Φ_k. Updating an artifact produced by a fixed procedure differs from revising the procedure that produces future artifacts. For example, a fixed trainer can improve a policy repeatedly, while an inherited change to data selection or candidate search can alter how later policy improvements are produced. More editable components do not necessarily imply a stronger recursive claim.

$$\delta_k \in \text{Propose}_{\Phi_k}(S_k, D_k), \;\; S_k^c = \text{Apply}(S_k, \delta_k) \tag{6}$$

Equation (6) separates an edit δ from its candidate system S^c. The editable set can include memory, harness programs, data generators, policies, world models, or improvement components. A proposal is not an accepted update.

$$S_{k+1} = \text{Select}_{E_{\text{val}}}(S_k, \{S_k^c\}; D_{\text{val}}) \tag{7}$$

Equation (7) makes acceptance explicit. Selection may retain the incumbent, and candidate admission can include interface, regression, and resource checks. E_{val} and D_{val} guide selection; a separate final evaluator and held-out tasks are needed to estimate generalization. A rejected candidate should remain part of the cost and provenance record.

Meta-learning is a related but distinct foundation. MAML learns an initialization that supports rapid adaptation using a specified gradient-based procedure [6]. This demonstrates optimization for subsequent learning, but does not imply that a deployed agent autonomously rewrites its optimizer or validates successive improver generations. Continual learning emphasizes adaptation while preserving prior competence. Automated search may optimize an agent externally. Embodied RSI can combine these approaches; its distinguishing questions concern who initiates and selects retained changes, which improvement mechanisms are inherited, and whether revised mechanisms improve later learning under comparable conditions.

PEARL illustrates task adaptation through probabilistic context inference and off-policy experience, providing a contrast to gradient-initialization meta-learning [18]. Both can improve adaptation while keeping the deployed adaptation procedure specified externally. POET jointly generates environments and their solutions, and unsupervised environment design studies useful training distributions [17,25]. These precedents motivate acquisition and curriculum mechanisms, while leaving the inheritance and independent meta-gain questions explicit.

$$\Phi_{k+1} = \text{MetaUpdate}(\Phi_k, \mathcal{A}_k; E_{\text{meta}}) \tag{8}$$

Equation (8) is this survey's abstraction of a meta-update. The archive $\mathcal{A}$ contains previous candidates, interventions, outcomes, costs, and rejections; E_{meta} evaluates a proposed change to the improvement procedure. A changed Φ must be invoked by a successor. Its effectiveness requires the matched comparison in Section 7.4; the equation asserts no convergence or monotonic-improvement guarantee.

Feedback first supports execution. Inner Monologue uses environmental feedback in planning, while Self-Refine revises an output through iterative feedback [43,58]. If the resulting change remains local to the current attempt, it is an execution or output improvement. Reflexion stores feedback for later attempts, and Voyager accumulates executable skills, illustrating persistent carriers outside model parameters [57,65]. Table 2 distinguishes these cases from evidence of an improved improver.

Table 2 Conceptual boundaries and required evidence

Concept	Retained change	Question for evaluation
Within-episode correction	Current plan, action, or output	Does the current execution improve?
Persistent self-improvement	Knowledge, code, policy, or model	Do later tasks benefit from this change?
Successive inherited improvement	Changes accumulated by successor versions	Do gains exceed additional search and experience effects?
Meta-level improvement	A proposal, acquisition, training, or selection procedure	Does a successor actually invoke the revised mechanism?
Demonstrated recursive gain	An inherited, more effective improver	Under matched resources and starting states, does it produce better subsequent improvements?

Internal success detectors, reward models, and selectors may themselves be improved, but final comparisons require an explicit reference outside the optimized mechanism. A rising best-so-far curve can arise from additional

search, a favorable initialization, or changing tests. It does not isolate a more effective Φ. Recursive gain should therefore be evaluated under matched starting systems, information, and budgets, including physical interaction and reset. Such gain is distribution- and resource-dependent; it does not establish indefinite acceleration.

Duan et al. organize B0 and L1–L5 around responsibility for in-task improvement, improvement execution, strategy selection, experience acquisition, deployment adaptation, and meta-improvement [133]. The Physical RSI framework instead distinguishes capability-level and meta-level updates, task scope, and independently evaluated recursive gain [160]. Equal level numbers do not have interchangeable meanings. We name the source framework whenever discussing a level, and retain autonomy and evidence as separate axes rather than combining them into a single maturity rank.

For robots, autonomy must be assigned to concrete decisions. Choosing the next experiment, selecting a policy for training, rejecting a candidate, deploying a revision, and stopping exploration are different responsibilities. Humans may retain the objective and resource authority while the agent optimizes within those boundaries. An accurate account discloses this allocation, the persistence carrier, the changed procedure if any, and the evaluation supporting inheritance.

Gao et al. [111] organize evolution by what, when, and how; Ren et al. [130] formalize self-induced updates to the model and operational scaffold. Chen et al. [129] additionally emphasize improvement targets, loop closure, and self-evaluation, while Wu et al. [151] distinguish infrastructure design from repair and optimization. Together these accounts motivate our four independent axes: update mechanisms and autonomy describe the process, while persistence, transfer, and matched-improver tests describe the evidence.

2.5 Reasoning Reflection and Memory in the Improvement Loop

The inference mechanisms in Section 2.1.2 organize alternatives; the memory and skill interfaces in Section 2.3 connect these alternatives to previous evidence and executable interventions. In an embodied improvement loop, reasoning can decompose a failed task, compare causes, choose informative observations, and propose edits. Reasoning quality should be judged by the interventions it enables and the evidence they produce, rather than by the coherence of an explanation alone.

Reflection interprets a completed or interrupted interaction in relation to the intended outcome. A useful reflection identifies the observed discrepancy, proposes possible causes, and specifies conditions under which a revised behavior should help. Self-Refine illustrates iterative output revision, and Reflexion retains verbal feedback for subsequent attempts [57,58]. In robotics, a statement such as “the grasp failed because the object moved” is a hypothesis until observations distinguish object motion from calibration error, occlusion, or an inadequate grasp policy. Re-execution, additional sensing, and component replacement can test it. Reflection can therefore guide experiment design, but cannot serve as its own physical verification.

$$K_{k+1} = \text{Consolidate}(K_k, \text{Validate}(\text{Reflect}(\tau_k))) \tag{9}$$

Equation (9) describes a functional memory-update pipeline. τ is an execution trace containing observations, invoked actions, and outcomes. Validate is evidence checking rather than a claim that all reflections are correct; an implementation may reject a lesson or retain it with uncertainty and applicability conditions. Consolidation may also retract conflicting entries.

Figure 2 connects these functions in an evidence-guided cycle: retrieval informs hypotheses, execution measures their consequences, and verification determines what can be consolidated. The return path through memory is what allows an accepted lesson to influence later tasks; the diagram does not assume that every generated reflection is correct.

Reasoning, reflection, and memory form a coupled mechanism. Retrieved experience informs a diagnosis; reasoning proposes a test or edit; tool use produces observations; reflection interprets the outcome; and a validated lesson changes future retrieval or behavior. Reasoning and reflection may operate entirely within an episode. Persistence appears when an accepted update survives beyond that episode, while meta-level change appears when the retained update modifies future diagnosis, acquisition, training, or selection. The same cognitive function can

therefore support execution, capability improvement, or meta-improvement depending on what is retained and used by a successor.

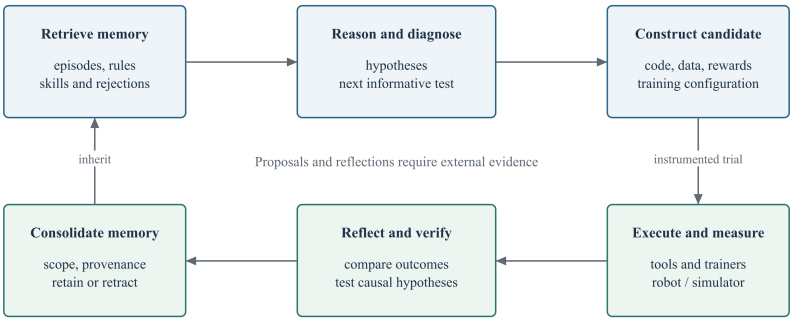


Figure 2 Reasoning reflection and memory

2.6 Large Model Guidance for Model and Platform Optimization

A large model can guide optimization as a proposer and experiment coordinator without directly implementing every numerical update. Its inputs may include goals, observations, execution traces, controller diagnostics, training curves, and retrieved experience. Its outputs can include a failure hypothesis, a requested measurement, a data-collection plan, a code patch, or a training configuration. The platform executes those artifacts through defined interfaces, and an acceptance process checks the resulting behavior. This separation makes "LLM-guided improvement" concrete: the review should identify the proposed artifact, the tool that applies it, the source of feedback, and the retained successor.

Toolformer studies learned API use, and ToolLLM develops tool-use instruction data and models [54,70]. SWE-agent shows that an agent–computer interface is itself a design variable, while AutoGen provides programmable multi-agent conversation patterns [71,90]. These works support analysis of platform interfaces, delegation, and tool authority. In a robot system, each interface must additionally identify the observations, actuation resources, timing, and physical state it can affect.

OPRO uses language models to propose optimization candidates; DSPy optimizes modular language-model pipelines; TextGrad propagates textual feedback through compound systems; and Automated Design of Agentic Systems searches agent designs [74,77,95,101]. They establish useful external-optimization precedents. An optimizer for an agent is not automatically an agent that improves its own optimizer: the identity, edit scope, inheritance, and later use of the improvement procedure must still be tracked.

For model optimization, the agent may select corrective examples, design curricula or rewards, configure a trainer, or choose which policy component to adapt. Eureka provides a reward-code example, and the embodied learning systems reviewed later supply policy and data-acquisition examples [81,112,149,152]. A language model can thus influence parameter learning through the objective, samples, or training procedure, while a numerical optimizer performs the weight update. For world models, the corresponding interventions may concern dynamics parameters, scene assumptions, or informative experiments [159]. Predictions and proposed corrections require downstream and reality-grounded validation.

For platform optimization, editable artifacts include context builders, perception queries, tool wrappers, skill composition, scheduling, recovery workflows, simulator configurations, and data-collection pipelines. The model, data, environment, and harness perspective helps locate these interventions within a broader robot system [162]. Here platform denotes the execution and learning infrastructure; a software edit does not itself demonstrate modification of robot hardware. Harness evolution provides examples of changing this infrastructure around existing models [140,144,147,150,157]. Calibration and controller settings can also affect behavior, but their interfaces and validation requirements differ from a prompt edit. The location of a change matters because model retraining, program revision, and physical calibration impose different costs and failure modes.

Consider a robot that repeatedly fails to place an object into a drawer. The agent can retrieve earlier trajectories and inspect whether the object reference, drawer state, approach pose, or motor behavior is responsible. It can

request an additional view, test an invocation change, collect corrective demonstrations, or revise a simulated dynamics assumption. Repeated failure alone does not justify choosing among these interventions. Candidate-specific tests and comparable starting conditions help determine which change addresses the cause. The accepted artifact and its applicability conditions then enter the successor system, while rejected hypotheses remain useful experimental records. This is an illustrative analysis of the improvement process rather than a reported experiment.

Hardware constraints affect which experiments an improver can afford. The hardware-oriented inference survey of Li et al. [102] compares platforms using speed, power, and energy efficiency. For robots, a faster model call helps only if sensing, communication, and control remain coordinated. Platform adaptation should therefore record the full observation-to-action delay and energy per completed task, as well as the cost of rebuilding, calibrating, or validating the platform after an update.

The resulting loop is evidence acquisition, diagnosis, intervention selection, candidate construction, validation, and inheritance. An improver may choose the next experiment by its expected contribution to reliable task gain relative to interaction, computation, reset, and supervision cost. This is a decision criterion, not a guarantee that an LLM estimates those quantities accurately. If the loop changes only an execution artifact, it supports capability self-improvement; if the inherited change revises how future experiments or candidates are produced and selected, it supports a meta-level claim. Controlled comparison is still needed to establish recursive gain.

$$\delta_k^* = \operatorname{argmax}_{\delta \in \Delta_k} \{\widehat{\Delta J}(\delta) - \lambda\, C(\delta)\}, \;\; R(\delta) \le \varepsilon \tag{10}$$

Equation (10) is a constrained decision criterion for choosing an intervention, not a proven estimator or an algorithm shared by all methods. Δ_k is the permitted edit set, estimated ΔJ is expected task gain, C is a scalarized resource cost with declared conversion weights, and R is a specified risk estimate. Risk estimation and any resulting safety claim require their own evidence; satisfying an estimated constraint is not a guarantee of physical safety [8].

3 Agent Harnesses and Embodied Agent Harnesses

3.1 Responsibilities and Physical Constraints

A general agent harness organizes the model's context, tools, memory, execution state, feedback, and control flow. Code as Agent Harness reviews code as a vehicle for connecting such mechanisms [123]. The embodied-harness survey emphasizes how these responsibilities extend through runtime governance and skill execution to hardware [142]. Its mechanism families cover grounding and composition, state and memory, scheduling and coordination, verification and recovery, and adaptation. This organization describes how execution is supported; our subsequent taxonomy describes how retained changes are produced and evaluated.

Operational and research harnesses serve related purposes. An operational harness governs a task attempt; a research harness supports collection, training, testing, and revision. The same software may participate in both roles. A harness becomes a self-improvement target when a retained revision changes future execution, and it becomes part of a recursive mechanism when an inherited revision changes future improvement. An operational harness with strong recovery can remain fixed throughout deployment.

Harness surveys expose different architectural views. He et al. [119] separate control, agency, and runtime; Meng et al. [120] specify an execution loop, tool registry, context manager, state store, lifecycle hooks, and evaluation interface. Li et al. [165] make observability and governance explicit in ETCLOVG, while Guo et al. [126] use observation, context, control, action, state, and verification. Huang et al.'s embodied responsibilities [142] add physical execution constraints. These decompositions can be mapped by responsibility, but their component counts are not maturity scores.

Table 3 compares the responsibilities of general and embodied harnesses. The physical column identifies why software retries and valid tool calls are insufficient evidence of successful robot execution.

Table 3 General and embodied harness requirements

Responsibility	General agent harness	Embodied agent harness
Grounding and interfaces	Tool schemas, files, APIs, executable programs	Objects, coordinate frames, affordances, robot skills and controllers
State and memory	Conversation, tool state, retrieved records	Uncertain scene state, motion history, geometry, calibration and skill versions
Scheduling	Calls, workflows, concurrent tasks	Multiple rates, actuator resources, deadlines and interruption
Verification	Tests, tool results, artifact checks	Observed postconditions, feasibility, authority and physical safety
Recovery	Retry, checkpoint, software rollback	Recovery motions and measured physical reset; software rollback is insufficient
Improvement evidence	Task tests, search and compute budgets	Physical trials, interaction and reset cost, wear and human assistance

3.2 Comparison of Harness Improvement Methods

General methods expose several transferable mechanisms. GEPA evolves prompts using trajectory reflection [110]. DGM and Hyperagents investigate editable agent implementations or improvement mechanisms, while AIDE² and Harness² provide further digital references for system search and inheritance [109,118,138,156]. These studies motivate candidate archives, trace-based diagnosis, controlled edit surfaces, and evaluation gates. Their digital results do not establish that the same changes are feasible, safe, or cost-effective on a robot.

Embodied methods specialize these ideas around execution interfaces and physical evidence. EnactOpt couples textual-skill revision with context code [157]. DynaHarness emphasizes capability invocation and regression checks [150]. RegenHarness emphasizes evidence-gated commitment and configuration traceability [140]. RoboFoundry evolves context and hierarchical skill systems [144]. AdaRSI connects harness updates to selective policy training when remaining failures require another intervention [163]. Their key differences concern what can be edited, which executors already exist, how feedback is obtained, and how revisions are accepted.

Table 4 compares representative harness methods by the artifacts they can edit and the evidence their claims require. It complements the mechanism discussion rather than providing a numerical ranking.

Table 4 Representative harness methods and improvement surfaces

Method family	Editable surface and mechanism	What the comparison must establish
ReAct and Code as Policies [45,47]	Reasoning/action traces and generated control programs	Execution composition; persistence and improvement require additional evidence
GEPA [110]	Prompt candidates guided by trajectory reflection	Search and evaluation budgets; transfer of retained prompts
DGM and Hyperagents [109,118]	Agent implementations or improvement mechanisms	Which procedure changes and whether successors use it
AIDE² and Harness² [138,156]	Digital agent search and inherited system variants	Artifact versus improver changes; independent downstream tests
EnactOpt [157]	Textual skills and context-construction code	Contribution of context versus skills under disclosed executors
DynaHarness and RegenHarness [140,150]	Invocation, governance, and evidence-based revision mechanisms	Existing capabilities, admission rules, recovery and regression effects

Method family	Editable surface and mechanism	What the comparison must establish
RoboFoundry [144]	Context and hierarchical skill systems	Behavioral expansion versus better orchestration
AdaRSI [163]	Harness revisions plus selected sub-step policy training	Attribution and added benefit relative to training cost
HexaAnything [145]	Physical coding; explicit state and policy; tools and trace-to-model updates	Which artifact improved; proposed co-evolution versus demonstrated updates
PhysEvo [155]	Task and meta harnesses; inherited diagnostic tools	Later repairs versus a matched causal test of improver gain
MotorMind [148]	Mid-level actions; asynchronous execution and memory	Operational adaptation versus inherited revisions
SoL-Pi [136]	Search for reusable execution and context mechanisms	Digital cost-quality trade-offs; embodied transfer untested
VLMeg [158]	Gated VLM feedback for robot RL	Policy learning and detector errors; improver revision not established

The comparison suggests a practical bridge rather than a shared leaderboard. Context assembly, program search, memory management, and candidate selection can transfer as algorithmic ideas. Embodied instantiation must add observation uncertainty, temporal contracts, actuator authority, recovery, and independent physical validation. Model-based simulation can reduce candidate cost, but acceptance requires accounting for model mismatch. We therefore compare mechanisms and experimental controls first; numerical comparisons are reserved for matched settings or explicitly qualified protocols.

MotorMind [148] links VLM-proposed mid-level actions to deterministic control, asynchronous monitoring, and background memory. It is a useful operational-harness comparator; same-episode adaptation does not by itself establish inherited improvement. SoL-Pi [136] instead searches reusable execution and context mechanisms in digital environments. Its efficiency evidence motivates testing cost-aware harness revision in robots, with physical effects evaluated separately.

4 A Four Axis Taxonomy

4.1 Improvement Targets

The target axis identifies what is modified and inherited. We use memory and skills, execution harnesses, data and environments, policy and model parameters, world models, and cross-layer or meta-level procedures as literature-navigation categories. Trainers, search procedures, evaluators, and feedback mechanisms receive separate fields in the coding scheme so that they are not hidden inside a broad system label. Improving an external task artifact counts as agent self-improvement only when it enters the successor system and affects later behavior or improvement.

Feng et al. [116] describe five embodied self-evolution components: memory updating, task switching, environment prediction, embodiment adaptation, and model evolution. These broaden the target space beyond policy weights. They map to several targets in Figure 3; they do not establish that every component is editable in every reviewed system or that an improved component changes the update procedure.

4.2 Update Mechanisms

The mechanism axis identifies how experience produces a change. We distinguish reflection and knowledge revision, executable-program search and evolution, adaptive experience acquisition, parameter learning, world identification and revision, and intervention selection or meta-procedure revision. These mechanisms can act on

different targets, and one system may combine several. Code is a representation, memory a persistence substrate, and evolution an update process; they should not be treated as equivalent classification units.

Table 5 maps each update mechanism to its typical modification and a comparison question. The same target can be modified through more than one mechanism, so its rows are not mutually exclusive system categories.

Table 5 Mechanisms and comparison questions

Mechanism	Typical change	Comparison question
Reflection and knowledge revision	Rules, conditions, and experience entries	Is the evidence relevant, and can incorrect knowledge be retracted?
Program search and evolution	Skills, context code, recovery workflows	What is searched, validated, and retained?
Adaptive experience acquisition	Demonstrations, curricula, scenes, tasks	Does acquisition provide useful information at acceptable cost?
Parameter learning	Policy, value, or auxiliary-model parameters	Are new capabilities acquired without losing old ones?
World identification and revision	Dynamics, explicit twins, predictive models	Do revisions improve downstream decisions and reality fit?
Selection and meta-procedure revision	Intervention targets, budgets, search and acceptance rules	Is selection fixed, or does the selection procedure itself change?

4.3 Decision Authority and Evidence Conditions

The autonomy axis records who controls diagnosis, intervention selection, acquisition, training, candidate acceptance, deployment, and stopping. The evidence axis records version changes, successor use, independent tests, transfer, physical trials, and controlled meta-gain comparisons. These axes remain separate: autonomous selection can rely on weak verification, while tightly bounded updates can yield reliable gains.

We describe each method by a primary mechanism, primary target, decision allocation, and evidence conditions, with secondary tags where needed. We do not reduce them to a single maturity score. Cross-layer updating is especially important to distinguish from meta-updating. A fixed rule can change memory, harness code, and policies without modifying Φ; revising a sampling or selection rule may change only one component while directly affecting future improvement.

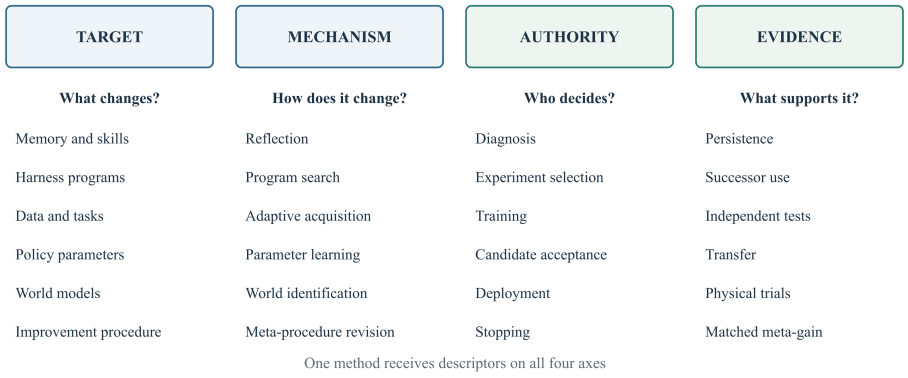


Figure 3 Four axes of embodied RSI

Figure 3 keeps the four classification axes separate. A method can modify several targets with one mechanism, or use several mechanisms on one target. Authority and evidence describe how those changes are governed and supported; they are not additional modules to count.

Nguyen et al. [132] distinguish self-learning, self-adaptation, self-optimization, and self-generation in humanoid self-evolution. These mechanisms complement our separate axes for targets, mechanisms, authority, and evidence.

In our interpretation, autonomy does not establish recursive gain: generating new experience or policies still requires a controlled test of whether the retained revision improves subsequent improvement.

5 Mechanisms for Embodied Self Improvement

5.1 Reflection and Knowledge Revision

Reflection transforms execution results into reusable descriptions of errors, successful conditions, object relations, and recovery strategies. Language and programs provide inspectable carriers that can change decisions without expensive backbone training. The difficulty is that a reflection may itself be wrong, and a useful lesson may depend on a particular scene or policy version. Reusable knowledge therefore needs applicability conditions and evidence provenance, not only an instruction describing what to do.

Voyager provides a digital-embodiment precedent for automatic curricula, skill accumulation, and feedback-based program repair [65]. RoboHarn-Evo separates task knowledge from action knowledge and revises their applicability using physical feedback [147]. CarryOn combines persistent preferences and scene context with recovery skills [164]. These systems show that knowledge can carry improvement, while leaving a separate question about whether the knowledge-management procedure itself improves.

Memory representations within an episode are relevant foundations rather than automatic evidence of RSI. EvoMem-VLA retains observed state changes to support long-horizon decisions [153]. Without additional persistent-update and inheritance experiments, such results establish the utility of memory representation rather than recursive improvement. Future comparisons should test retraction, scope restriction, competence preservation, and retrieval under interfering experience.

5.2 Executable Programs and Harness Evolution

Program evolution exposes skill implementations, context assembly, and control flow as editable artifacts. A typical loop diagnoses a trajectory, proposes a change, checks interface compatibility, evaluates a candidate, and retains a successor. Unlike a textual rule alone, executable code can change what evidence reaches the model, how tools are invoked, and when execution stops or recovers. Its effects may span perception, planning, and control.

EnactOpt jointly revises textual skills and context-construction code, addressing the coupling between missing evidence and inadequate use of available evidence [157]. DynaHarness coordinates capability invocation through execution contracts, fast and slow loops, and regression checks [150]. RegenHarness emphasizes evidence-gated commitment and configuration traceability [140]. RoboFoundry treats context and hierarchical skill systems as an evolving policy [144]. Comparisons should disclose the editable surface, admission checks, available executors, and tool authority, rather than count modules or compare architecture names.

This route is constrained by both the edit space and available behavior. Better invocations can repair target references, initial positioning, ordering, or recovery logic, but cannot guarantee a missing motor capability. Conversely, newly generated executable skills can expand the behavior library even when backbone weights remain frozen. Experiments must distinguish runtime governance, added skills, and the update search that produced them.

General optimizers also provide useful controls. GEPA uses trajectory reflection for prompt evolution [110]; comparing such a baseline with an embodied-specific optimizer helps distinguish generic search benefits from domain-specific diagnosis. These comparisons still require disclosure of interaction and candidate-evaluation budgets.

HexaAnything [145] makes task state and control explicit as Code as World and Code as Policy; verified traces can support tools and model training. We treat these as distinct update surfaces. The report demonstrates partial improvements; its proposed architecture, embodiment, and hardware evolution roadmap should not be counted as completed autonomous co-evolution.

5.3 Adaptive Experience and Environment Coevolution

Embodied experience depends on the current policy. Early failures limit which later states are reached, creating uneven opportunities for learning. Adaptive acquisition changes tasks, starting states, demonstrations, or corrective continuations to concentrate resources near the current capability frontier. Its objective is not simply more data, but informative experience that supports an effective intervention.

DAgger addresses the distribution shift caused by a learner's own actions through iterative imitation-learning data aggregation [4]. Hindsight Experience Replay reuses trajectories under relabeled goals [11]. They explain why a robot's failures and visited states matter for subsequent training, but their original procedures do not imply autonomous revision of a trainer. Dex-Net 2.0 provides a synthetic-grasp-data precedent [7]; its analytic grasp labels illustrate why supervision and privileged assumptions must be traced separately from real execution feedback.

EmbodiRSI uses corrective trajectories and adaptive collection within a real-to-sim-to-real pipeline [149]. EmbodiedSmith jointly revises scenes and tasks to expand simulated experience [154]. EmbodiedSWE converts solutions found by coding agents into supervision for robot policies [139]. These methods may improve samples, sample generators, or both; their implications for recursion differ.

Rewards and curricula are also editable targets. Eureka searches reward code using training feedback [81]. If reward search, task admission, and training remain fixed, the observed gain primarily supports better objective or data design rather than an improved meta-procedure. Evaluation should account for failed generation, infeasible scenes, invalid demonstrations, and validation costs, and test data value through independent downstream outcomes.

Human video provides another acquisition route. Feng et al. [124] distinguish latent actions, predicted futures, 2D cues, and 3D reconstruction as transferable supervision. Our comparison records whether an agent merely consumes a fixed video corpus or changes which demonstrations it acquires, how it grounds them, and how their utility is tested. Embodiment mismatch makes video abundance an insufficient proxy for effective robot experience.

5.4 Parameter Learning and Autonomous Practice

Parameter updates can acquire motions that existing executors do not provide. Self-Improving Embodied Foundation Models combines post-training with autonomous practice [112]. MobiAgent segments, verifies, and recycles deployment experience for skill-policy learning [152]. EmbodiRSI connects targeted simulated practice to physical adaptation [149]. In these systems, outcomes generated by a policy supply training evidence for its successor.

Compared with external knowledge updates, parameter learning typically requires greater interaction and training expenditure. It also faces incorrect pseudo-labels, changed state distributions, and forgetting. Evaluation must include previously solved tasks as well as newly acquired behavior. When privileged simulator state generates expert demonstrations, its training-time contribution must be separated from observations available at deployment.

LoRA and QLoRA provide parameter-efficient adaptation mechanisms, with QLoRA combining quantization and low-rank adapters [27,64]. Their relevance is the cost and scope of an update, rather than a demonstrated guarantee of robotic adaptation. A policy update should disclose which weights or adapters change, which modules remain frozen, and whether memory or context changes are also contributing.

$$\mathcal{L}_{\mathrm{BC}}(\theta) = -\mathbb{E}_{(h,a)\sim D_{\mathrm{imp}}} \log \pi_\theta(a \mid h) \tag{11}$$

Equation (11) is a standard conditional behavioral-cloning objective for a probabilistic action model, where h is an observation/action history or its representation. Deterministic, diffusion, and flow policies use their appropriate training losses rather than necessarily this exact likelihood. The improver can alter the data and training configuration without directly calculating the gradient. This separates language-guided training decisions from the numerical optimizer [48,56,62,69,92,103].

Continual reinforcement or imitation learning does not inherently establish meta-level recursion. If the objective, algorithm, acquisition rule, and acceptance procedure are fixed, the result is primarily policy self-improvement. A further question is whether the agent can revise those decisions using previous training outcomes, retain the revised procedure, and apply it effectively to a new task.

VLMeg [158] gates episodic VLM success judgments and adds video-guided exploration for robot RL. It demonstrates feedback-driven policy learning, with the gate and learning recipe declared; it does not automatically demonstrate an improved improver. Success-detector false positives and human intervention must be evaluated separately from the policy reward.

5.5 World Identification and Model Revision

World models can be improvement targets and services for generating or evaluating experience. D-World revises an explicit digital twin using measured residuals [159], demonstrating an intervention surface in the assumptions about the world. DreamerV3 provides a foundation for learning through modeled experience [51], while DreamZero illustrates coupled future and action modeling [117]. Their roles should be identified through the actual execution and learning interfaces.

The World Models formulation learns compressed environment representations; PlaNet performs planning with latent dynamics; Dreamer learns behavior through latent imagination; and TD-MPC2 uses an implicit world model for continuous-control planning [14,16,21,82]. DayDreamer brings world-model learning to physical robots [42]. Together these works show that a world model can support either planning or learning. Their roles must be distinguished from changing the procedure that acquires and validates improved world models.

$$\mathcal{L}_W(\psi) = -\mathbb{E}_{(h_t,a_t,o_{t+1})\sim D} \log p_\psi(o_{t+1} | \, h_t, a_t) \tag{12}$$

Equation (12) is an illustrative observation-prediction objective. ψ parameterizes a model of the world. Latent, task-oriented, and implicit models can use different losses, so this equation is not imposed on every system [14,16,21,42,82]. Evaluation must separately test prediction, decision utility, and physical outcomes; minimizing a self-generated prediction loss alone does not establish useful improvement.

Current-state estimation, persistent dynamics calibration, and revision of a model-learning procedure are different changes. State estimation may not survive an episode; improved prediction may persist without changing Φ. Stronger RSI evidence connects a model revision to more useful experience, better candidate selection, or improved physical behavior, while avoiding evaluation solely inside the same model that generated the experience.

Visual realism is only one world-model property. Task-relevant geometry, action conditioning, long-horizon errors, and downstream utility may behave differently from perceptual quality. World-model-supported improvement should therefore measure both prediction and functional outcomes, controlling the contributions of action extraction, policy training, and reality-grounded feedback.

Physics priors provide another model-revision dimension. Piccinini et al. [135] distinguish priors embedded in inputs or data, architecture, and training losses. An embodied improver could select among these interventions, but their effects need independent physical validation. A lower prediction error may coexist with worse contact behavior if an imposed prior is inappropriate for the objects or operating regime.

5.6 Cross Layer Selection and Meta Improvement

Cross-layer coordination concerns matching an intervention to a failure, rather than maximizing the number of edited components. AdaRSI first improves the harness, retries persistent failures from matched simulator states, and trains selected sub-step policies when harness revisions do not suffice [163]. It brings invocation problems and executor limitations into a common framework and makes training expenditure an explicit decision variable.

Such selection may still follow a fixed procedure. DGM, Hyperagents, AIDE², and Harness² provide digital references for self-modifying implementations, editable improvers, or two-level inheritance [109,118,138,156].

Physical transfer adds measurement noise, irreversible actions, reset requirements, and hardware constraints. Success in digital tasks alone does not demonstrate a reliable robotic meta-improver.

The distinction also clarifies earlier autonomous-data and language-model bootstrapping. RoboCat uses its trained agent to generate data for subsequent training rounds [68]. STaR bootstraps reasoning supervision, and Self-Instruct generates instruction data for model adaptation [36,50]. These systems make different artifacts persist. A fixed data-generation and training recipe can support self-improvement without revising the recipe that produces future gains.

Figure 4 summarizes intervention selection across memory, harnesses, data, policy parameters, and world models. Diagnostic tests determine which branch is worth trying, while independent validation governs acceptance; several branches may be combined.

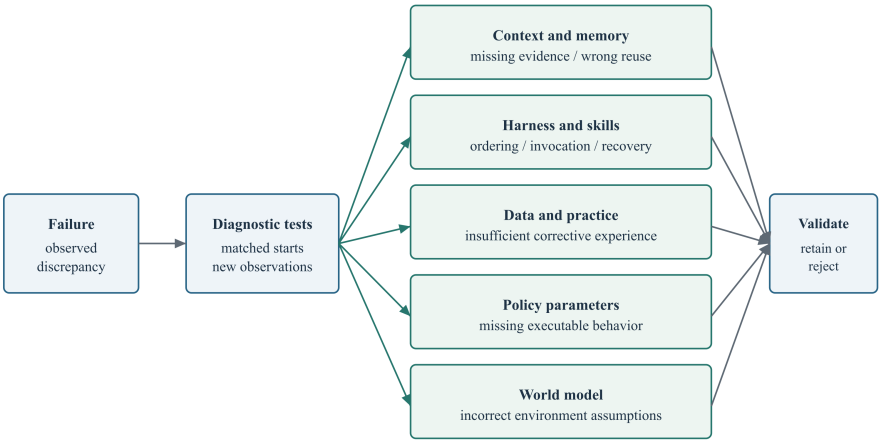


Figure 4 Evidence guided intervention selection

A meta-update should identify whether the changed component is a diagnostician, sampler, trainer, search strategy, or selector, and verify its use by a successor. Old and revised procedures should then start from comparable systems and evidence under matched budgets. A rising best-so-far curve cannot by itself separate a better improver from more experiments or a stronger initialization.

PhysEvo [155] permits a meta-agent to revise both task tools and its own diagnostic resources around a frozen model. Inherited review tools support later repairs, providing a concrete meta-level mechanism. Our assessment still requires a matched comparison of revised and baseline improvers to quantify the causal gain from those resources.

6 Comparative Performance Evidence

Task definitions, backbones, action interfaces, and improvement budgets vary substantially across current systems. We use reported numbers to characterize demonstrated changes, rather than create a common ranking. Each result retains its original baseline and evaluation setting; primary comparisons and component ablations must remain distinct.

Table 6 presents reported before-and-after results together with the limits of their interpretation. Within-row improvements are meaningful under the cited protocols; values across rows do not establish a common leaderboard.

Table 6 Reported results and interpretation limits

Method	Reported result	Interpretation limit
EnactOpt [157]	Initial harness to final system: VLABench 47.39% to 57.83%; ALFWorld 66.06% to 92.70%; EAI 72.34% to 95.74%	These use H_0 as the reference; comparisons are not compute-matched.

Method	Reported result	Interpretation limit
AdaRSI [163]	Unseen composite tasks: VLA 4.6%; harness 9.9%; complete system 13.8%	The weight stage adds 3.9 points over the harness; the cited version evaluates simulation.
EmbodiRSI [149]	Simulation 50.4% to 83.5%; physical result 83.1% versus 75.0%	Interpret alongside different simulated and real demonstration recipes.
DynaHarness [150]	Frozen policy 17.5%; complete system 75.2% on newly sampled starts	The full gap does not isolate recursion; skill and governance effects require controls.
MobiAgent [152]	RoboCasa 7.5% to 27.5%; Astribot S1 32.5% to 57.5%	Platforms and task sets differ; policy gain is not a direct meta-gain measure.
CarryOn [164]	Violation rate 52.6% to 6.1%; recovery success 13% to 80%	Safety and recovery outcomes are not overall task success.
PhysEvo [155]	RoboDojo: 62.00% success under five-dimension weighting; 58.57% under equal-task weighting	Task-specific retained versions; these aggregates differ
HexaAnything [145]	RoboCasa365: Composite-Unseen 34.3% to 38.3%; overall 56.6% to 61.1%	Same XR-1 action model; harness effect differs from meta-gain
MotorMind [148]	LIBERO-PRO: 66.7% base and 53.8% perturbed success	Zero-shot action interface; not a recursive-gain measurement
SoL-Pi [136]	EdgeBench efficiency point: 49.0% less token traffic; score 42.0 versus Pi 44.8	Digital benchmark; efficiency point trades off score

Negative evidence is equally informative. The report on 123 rounds of agentic skill discovery describes local changes that repeatedly passed tests without completing the target task, exposing relational perception, early-step concentration, and evaluation weaknesses [141]. Improvement rounds, accepted edits, and skill-library size cannot replace end-to-end outcomes or independent transfer tests.

Interpretation should separate three comparisons. Execution-level tests isolate invocation and recovery with fixed capabilities. Capability-level tests measure retained changes on later tasks. Meta-level tests compare the subsequent improvement efficiency of old and revised Φ. One system may offer strong evidence at one level while leaving another unresolved.

7 Benchmarks Datasets and Evaluation Protocols

7.1 Distinguishing Benchmarks Data and Baselines

A benchmark defines tasks, environments, protocols, and metrics; a dataset supplies demonstrations, trajectories, assets, or evaluation instances; a baseline is a compared method. A common benchmark name does not ensure a common protocol, and an official resource’s total size is not the amount used by a particular method. Table 8 summarizes resource roles, while Table 9 records selected method-specific partitions.

Additional established resources widen the evaluation coverage. CALVIN targets language-conditioned long-horizon manipulation, RLBench provides manipulation tasks and demonstrations, and ManiSkill2 emphasizes generalizable manipulation skills [20,32,53]. Habitat supports embodied-agent simulation, AI2-THOR provides interactive indoor environments, iGibson 2.0 emphasizes object-centric household simulation, ProcTHOR generates environments procedurally, and VirtualHome represents household activities as programs [13,15,19,28,41]. These resources support different action abstractions and data-generation processes; they should not be pooled into a single difficulty scale.

Table 7 summarizes additional platforms by the role of their tasks and data. The final column identifies protocol information needed before using them to assess improvement.

Table 7 Additional platforms and benchmark data roles

Resource	Tasks and data role	RSI evaluation issue
CALVIN [32]	Language-conditioned task sequences and interaction demonstrations	Test sequence composition and scene/task splits.
RLBench [20]	Manipulation tasks and demonstration generation	Keep task variations and demonstration access explicit.
ManiSkill2 [53]	Manipulation environments, assets, and demonstrations	Disclose geometry variation and privileged state.
Habitat [19]	Agent/sensor simulation and supported scene datasets	Name scene dataset, embodiment, and action interface.
AI2-THOR and ProcTHOR [13,41]	Interactive scenes and procedurally generated houses	Separate house generation from hidden evaluation.
iGibson 2.0 [28]	Object-centric household simulation	Report supported physical processes and reset conditions.
VirtualHome [15]	Programs and simulated household activities	Program executability differs from continuous robot control.
RoboDojo [155]	42 simulation tasks; five held-out layouts each	Task-specific versions; dimension versus task weighting
MetaWorld and Franka [158]	12 simulation and 3 physical tasks; expert video supervision	Human interventions, detector error, and learning budgets
EdgeBench [136]	51 public digital tasks; 11 acceptance and 40 final-evaluation tasks	Acceptance versus held-out subsets; report cost with score

Table 8 connects the main evaluation benchmarks to their associated data and required disclosures. Resource availability should be distinguished from the subset actually used by a reviewed method.

Navigation broadens the evaluation scope beyond tabletop manipulation. Zhang et al. [98] review foundation-model-based vision-and-language navigation. Route progress, instruction grounding, collision exposure, and recovery should accompany terminal success. A navigation harness that replans successfully within one route still needs cross-route inheritance and independent tests before supporting an RSI claim.

Table 8 Benchmarks and associated data

Benchmark	Associated data and evaluation target	Conditions to disclose
RoboCasa365 [166]	Kitchen assets, tasks, and demonstrations; target suites contain 18 atomic, 16 seen composite, and 16 unseen composite tasks, with 500 demonstrations per task.	Checkpoint-relative seen status, target scenes, horizons, and version.
LIBERO [167]	Language-conditioned manipulation data and Spatial, Object, Goal, and 100-task suites.	Suite, demonstrations, task sequence, retention protocol.
LIBERO-PRO [168]	LIBERO-derived task and initial-state files with configurable perturbations.	Perturbation type, strength, combinations, starts, and implementation version.
VLABench [169]	Manipulation assets, task configurations, and demonstration resources; execution and higher-level reasoning evaluation.	Task and track subset; closed-loop success versus sequence matching.
ALFWorld [170]	PDDL, game files, and trajectory metadata; 3,553 training games and 140/134 seen/unseen validation games.	Text versus visual execution; official validation versus custom development splits.
EAI [171]	BEHAVIOR/VirtualHome states, goals, and action-sequencing resources.	Module, scene, executability, and goal-satisfaction definition.
EmbodiedBench [172]	Four environments with higher- and lower-level tasks and capability tests.	Environment subset, action abstraction, capability breakdown.

Benchmark	Associated data and evaluation target	Conditions to disclose
BEHAVIOR-1K [173]	Activity definitions and scene/object assets; challenge releases provide separate task and demonstration sets.	Activity subset, simulator version, complete success versus partial credit.
RMBench [174]	RoboTwin-based memory-dependent manipulation tasks and resources.	Dataset identity, release, and memory conditions.
RoboMME [175]	Sixteen tasks in four suites and 1,600 training demonstrations, with fixed validation/test episodes.	Continuous actions, waypoints, and multiple-choice interfaces are distinct.
EWMBench [176]	Reference videos, initial frames, and descriptions for generated-world evaluation.	Reference versus generated samples; video scores do not replace interaction tests.
WorldArena [177]	RoboTwin2.0 Clean-50 data and perceptual/functional evaluation.	Release, downstream policy, action extraction, and training budget.

7.2 Data Roles in an Improvement Loop

We distinguish six roles: D_{pre} for backbone pretraining; D_{seed} for initial adaptation or demonstrations; D_{imp} for improvement experience; D_{val} for candidate acceptance; D_{test} for final evaluation; and D_{transfer} for meta-procedure transfer. They may originate in the same environment but should be separated by instances, scenes, objects, or task families where the claim requires it.

Open X-Embodiment, DROID, and BridgeData V2 can document robotic priors available to a backbone [73,79,88]. Their presence in a specific method must be established from its actual recipe, rather than inferred from model family. Aggregated datasets may overlap constituent datasets and should not simply be added together. Static demonstrations support behavior learning, whereas RSI also needs evidence about failures, rejected candidates, recovery, and physical reset.

The data-pyramid perspective similarly emphasizes that data sources differ in their relationship to robotic behavior, rather than only in size [131].

RoboMimic studies how dataset and learning choices affect imitation from offline human demonstrations, and RLDS provides an ecosystem for recording and using sequential-decision data [29,31]. For RSI, reproducible data handling should additionally retain candidate identifiers, reset records, rejections, human interventions, and the configuration under which each trajectory was collected. An environment asset set, a demonstration corpus, and a final test partition serve different evidential roles.

Task counts, trajectory counts, action frames, and physical interaction hours measure different resources. Reporting only retained successful trajectories omits failed exploration and candidate evaluation. A complete account separates total collection, retained experience, training samples, privileged labeling, and human assistance. This is necessary to interpret claims of data efficiency or low supervision.

VLM benchmark surveys [106] cover diverse spatial, temporal, and embodied evaluations. Such scores are useful for selecting a perception or reasoning backbone, but cannot replace closed-loop tests of the coupled robot system. Upgrading a VLM and changing the harness simultaneously requires separate backbone, harness, and interaction ablations.

7.3 Method Specific Partitions and Metrics

Table 9 records method-specific development, selection, and evaluation partitions. It shows why official benchmark totals cannot substitute for the experimental sample counts of individual papers.

Table 9 Data use in representative methods

Method and benchmark	Improvement and selection	Final evaluation and scope
EnactOpt VLABench [157]	30 development and 30 validation instances; adaptation uses T1.	920 test instances spanning T1/T3/T4/T6; not the full benchmark size.
EnactOpt ALFWorld [157]	30 development and 30 validation instances drawn from official training.	All 140 valid_seen and 134 valid_unseen instances used as final evaluation.
EnactOpt EAI [157]	30 development and 30 validation instances, with related instances grouped.	282 tests from VirtualHome scene_1; not cross-scene evaluation.
AdaRSI RoboCasa365 [163]	Harness feedback and self-collected demonstrations; separate development, validation, and test scenes.	50 target tasks with 50 evaluation scenes per task; no privileged deployment observations.
EmbodiRSI [149]	Targeted simulated correction and collection, followed by physical refinement.	Fourteen subtasks in deployment-specific settings; separate tests are needed for workspace transfer.
PhysEvo RoboDojo [155]	Development layouts support revision; retain a version per task	Five excluded layouts per task; reset writable memory; fixed scored version
HexaAnything RoboCasa365 [145]	Fixed XR-1 versus coding harness; trace-based model development is a separate intervention	50 tasks, 50 seeds each; disclosed benchmark split, not all 365 tasks
VLMeg MetaWorld [158]	3–5 expert videos per task; generator adaptation and online gated reward	300k steps, assembly 1M; disclose differing baseline RL backbones
SoL-Pi EdgeBench [136]	Mechanism search excludes evaluation outputs; freeze candidate rules	11 acceptance tasks and 40 final tasks; pooled public-set scores need this qualification

RoboCasa365's Composite-Unseen label is relative to its pretraining distribution: target demonstrations for these tasks are still provided [166]. Claims of unseen-task generalization must identify the training stage and checkpoint. Official documentation also records a horizon change in version 1.0.1. Horizon length, tool-step accounting, and termination rules can affect success, so current defaults should not be assumed to describe historical experiments.

Metrics should follow the claim. Manipulation evaluation includes full success, partial goals, and stage reachability. Recovery adds detection delay, recovery rate, and intervention cost. Memory evaluation requires history ablations, incorrect reuse, and retention. World-model evaluation needs prediction and functional utility. Within-episode memory scores do not alone establish cross-episode learning, and video-generation quality does not alone establish policy or improver gains.

7.4 Controlled Evaluation of Recursive Gain

Existing benchmarks provide execution and generalization tests, but their use alone does not validate recursive gain. We recommend continuous task streams, persistent version inheritance, fixed-Φ controls, separate acceptance and hidden evaluation, and records of both rejected and deployed candidates. Tests with a frozen final system should be distinguished from tests that permit further online adaptation.

$$\boldsymbol{b} = (b_{\text{tokens}}, b_{\text{GPUh}}, b_{\text{physical}}, b_{\text{reset}}, b_{\text{human}}) \tag{13}$$

Equation (13) records budgets as a vector with declared units: model tokens, GPU hours, physical interactions, reset time, and human effort. A scalar cost, if used in Equation (10), requires explicit conversion weights. Equal episode counts alone need not represent equal resource use.

$$S^{\pm} = \text{Improve}(S, D; \Phi^{\pm}, \boldsymbol{b}), \;\; G_{\text{meta}}(\boldsymbol{b}) = \mathbb{E}[J_E(S^{+}) - J_E(S^{-})] \tag{14}$$

In Equation (14), Φ^{+} is the revised improver and Φ^{-} the baseline; both start from the same S and D under matched budgets. J_E is the score of an independent final evaluator on tasks excluded from candidate selection, and the

expectation is over tasks and random seeds. Paired differences and uncertainty estimates support a distribution-specific claim. This operational definition does not demonstrate unbounded acceleration.

For lifelong evaluation, Nguyen et al. [132] propose adaptation rate, forgetting rate, transient cost, and safety regression rate. These trajectory metrics complement our matched test of improver gain.

Figure 5 compares revised improvers from the same initial system and experience under comparable budgets. Selection uses validation data; final evaluation uses separate tasks and a common independent evaluator. This design separates improver gain from a stronger inherited system or additional search and training.

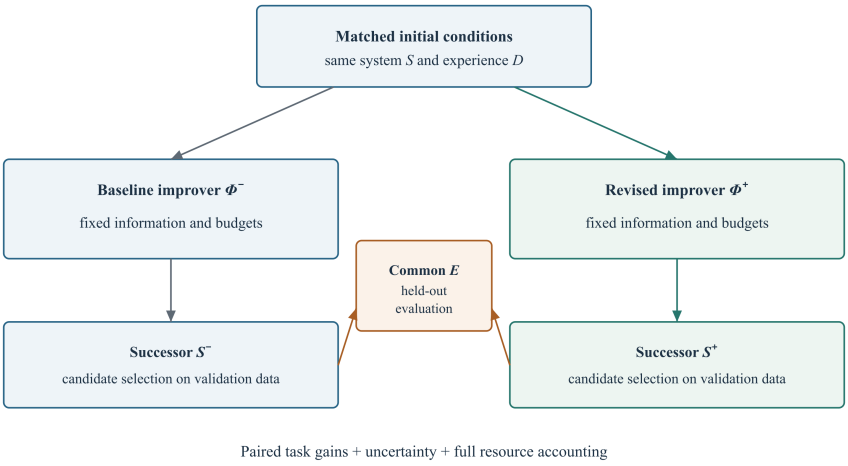


Figure 5 Matched comparison of improvers

Numerical comparisons should be stratified. Matched backbones, checkpoints, initial states, tool authority, budgets, and evaluators support direct differences. A shared benchmark with different protocols supports qualified comparison only. Text actions, continuous control, physical custom tasks, and different outcome metrics primarily support mechanism-level comparison. Reports should distinguish task-macro from episode-micro averages and include paired conditions, multiple seeds, and task-level uncertainty.

8 Open Problems and Future Directions

8.1 Causal Failure Attribution

The same final failure can arise from different components. Controlled fault injection, matched-state retries, and component replacement can test whether a diagnosis identifies an effective intervention. Attribution accuracy should be accompanied by final performance and cost under equal budgets. Systems should preserve uncertainty: failure to find a better invocation within a limited search is not proof that the executor can never perform the motion.

8.2 Reliable Feedback and Evolving Evaluators

Related models may propose actions, explain failures, and judge success, producing correlated errors. Feedback should be connected to traceable trajectories, independent observations, and observable postconditions. Evaluators may improve, but external anchors are needed to preserve comparability and expose exploitation. Task completion, invocation authority, and physical safety are distinct propositions and require different evidence [142].

Human-feedback instruction tuning, Constitutional AI, and Self-Rewarding Language Models differ in how supervision and evaluators are produced [34,49,84]. They motivate explicit accounting for human principles, preference judgments, and model-generated feedback. Their language-alignment outcomes do not demonstrate physical safety or evaluator independence. In embodied RSI, an evolving judge should be checked against observations and outcomes outside the mechanism being optimized. Constrained Policy Optimization provides a decision-theoretic precedent for keeping rewards and constraints distinct [8].

He et al. [99] analyze agent security and privacy; Kim et al. [125] distinguish planning-, policy-, and execution-time safety and separate formal guarantees, statistical evidence, and heuristics. For embodied RSI, these questions apply to both each accepted version and the process admitting changes. Gong et al. [143] extend safety evaluation across states, updates, trajectories, and lineages. We therefore propose tracking delayed regressions and dependent artifacts after a rollback, rather than assessing each candidate in isolation.

8.3 Physical Reset and Safe Inheritance

Restoring software does not restore physical state. Failed resets change the initial conditions of subsequent experiments, and hidden human rearrangement can overstate learning autonomy. Software promotion and physical reset should therefore be tracked as coupled but separately verified processes. Reports should include reset success, elapsed time, state drift, and human effort. Updated skills also require renewed checks of entry conditions, resources, termination, and interruption behavior.

8.4 Experience Coverage and Competence Preservation

Long tasks often fail early, depriving later skills of practice. Stage-specific starting states, corrective continuations, and reverse curricula can improve coverage, but their reset and privileged-state advantages must be disclosed. Growing libraries also require controls for knowledge pollution, retrieval interference, forgetting, and regressions. Incorrect experience should be retractable or restricted to a narrower applicability domain.

Elastic weight consolidation and Gradient Episodic Memory provide distinct continual-learning precedents: parameter regularization and episodic-memory-based constraints on updates [5,10]. Their assumptions and evaluations differ from open-ended robotic deployment. They motivate reporting both acquired competence and retained competence, together with replay or regularization cost.

$$F = \frac{1}{|\mathcal{T}_{\text{old}}|}\sum_{g\in\mathcal{T}_{\text{old}}}\max\{0, J_g(S_{\text{old}}) - J_g(S_{\text{new}})\} \tag{15}$$

Equation (15) defines a simple regression measure over a fixed set of previously evaluated tasks; it is this survey's operational summary, not a universal forgetting metric. J_g uses the same protocol for both versions. F should be reported beside new-task gains, transfer, uncertainty, and replay expenditure; truncating at zero prevents improvements on some old tasks from hiding regressions on others.

8.5 World Model Coevolution and Reality Grounding

Progress inside an improving simulator may reflect adaptation to that simulator's errors. Independent physical residuals, held-out conditions, and uncertainty-directed experiments can distinguish modeled progress from real gains. Experiments should test whether generated experience, policy rankings, and exploration recommendations are useful, while isolating confounds introduced by action decoders and inverse-dynamics models.

The body-aware verification perspective of Nguyen et al. [132] motivates testing a learned verifier under physical drift; a proposed gate alone does not establish safe recursive improvement.

8.6 Transferable Improvers and Stopping Decisions

Whether a stronger execution system produces a stronger Φ requires separate causal evidence. Freezing S, swapping diagnosticians or samplers, and transferring a fixed revised Φ to new tasks can identify the source of meta-gain. Stopping is also an improvement decision: when expected benefit falls below cost or physical risk, the system should stop, acquire new evidence, or switch intervention layers. Saturation curves and rejected-update records make these limits visible.

Cross-embodiment transfer should distinguish interface reuse, implementation reuse, and behavioral transfer. A shared skill name or API does not make motor behavior portable. Calibration and adaptation budgets on a new

robot should be controlled, and initial performance reported separately from subsequent learning speed. Claims of general improvement knowledge are stronger when revised procedures remain useful under these changes.

9 Discussion

The allocation of responsibility among models, harnesses, and improvers is empirical rather than fixed. A stronger VLA may internalize memory or planning, while an external harness may add executable behaviors. Our decomposition supports causal analysis and does not prescribe a universal robot software stack. A method should be classified through its actual editable artifacts and decision authority.

Autonomy and evidence should not be compressed into one maturity ranking. Bounded updates can yield reproducible gains; broader self-modification can introduce validation overfitting, memory errors, and expensive exploration. Digital studies of regularized harness evolution and evidence-governed search suggest that constraining edits and improving search organization may matter more than expanding permissions [137,161]. Their physical effects require embodied tests.

Recent general and physical RSI surveys already discuss the boundary between capability updates and meta-improvement [133,160]. A dedicated embodied survey should add intervention-level analysis: connect failure types to modifications, trace supervision and data use, and assess each result under comparable conditions. Missing experimental details should remain explicit evidence gaps rather than be replaced by taxonomy labels.

Jiang et al. [128] view deployment improvement as converting traces into skills, memory, environmental feedback, parameter consolidation, and meta-level control. This complements our mechanism review: a runtime can be both the object revised and the infrastructure that decides which experience becomes evidence. The resulting research question is when fast external revisions should be consolidated into slower model updates without losing traceability or older competencies.

10 Conclusion

Embodied self-improvement now extends beyond a single policy to knowledge, skills, harnesses, data, and world models. The four-axis framework in this survey separates what changes, how it changes, who controls improvement, and what evidence supports the result. Existing studies demonstrate useful component-level and system-level gains, but iteration count, frozen backbones, or higher final success do not establish recursive gain by themselves. Reliable attribution, complete physical-cost accounting, competence preservation, and independent improver comparisons are needed to determine whether robots can improve not only current behavior but also the acquisition of future capabilities.

References

[1] Kaelbling LP, Littman ML, Cassandra AR. Planning and acting in partially observable stochastic domains. Artificial Intelligence, 101(1–2), 99–134, 1998. https://doi.org/10.1016/S0004-3702(98)00023-X

[2] Sutton RS, Precup D, Singh S. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. Artificial Intelligence, 112(1–2), 181–211, 1999. https://doi.org/10.1016/S0004-3702(99)00052-1

[3] Schmidhuber J. Goedel Machines Self-Referential Universal Problem Solvers Making Provably Optimal Self-Improvements. arXiv:cs/0309048, 2003.

[4] S Ross, et al. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. arXiv:1011.0686, 2010.

[5] J Kirkpatrick, et al. Overcoming catastrophic forgetting in neural networks. arXiv:1612.00796, 2016.

[6] Finn C, Abbeel P, Levine S. Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks. arXiv:1703.03400, 2017.

[7] J Mahler, et al. Dex-Net 2.0: Deep Learning to Plan Robust Grasps with Synthetic Point Clouds and Analytic Grasp Metrics. arXiv:1703.09312, 2017

[8] J Achiam, et al. Constrained Policy Optimization. arXiv:1705.10528, 2017.

[9] Vaswani A, et al. Attention Is All You Need. arXiv:1706.03762, 2017.

[10] D Lopez-Paz, et al. Gradient Episodic Memory for Continual Learning. arXiv:1706.08840, 2017.

[11] M Andrychowicz, et al. Hindsight Experience Replay. arXiv:1707.01495, 2017.

[12] Schulman J, et al. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.

[13] E Kolve, et al. AI2-THOR: An Interactive 3D Environment for Visual AI. arXiv:1712.05474, 2017.

[14] D Ha, et al. World Models. arXiv:1803.10122, 2018.

[15] X Puig, et al. VirtualHome: Simulating Household Activities via Programs. arXiv:1806.07011, 2018.

[16] D Hafner, et al. Learning Latent Dynamics for Planning from Pixels. arXiv:1811.04551, 2018.

[17] R Wang, et al. Paired Open-Ended Trailblazer (POET): Endlessly Generating Increasingly Complex and Diverse Learning Environments and Their Solutions. arXiv:1901.01753, 2019.

[18] K Rakelly, et al. Efficient Off-Policy Meta-Reinforcement Learning via Probabilistic Context Variables. arXiv:1903.08254, 2019.

[19] M Savva, et al. Habitat: A Platform for Embodied AI Research. arXiv:1904.01201, 2019.

[20] S James, et al. RLBench: The Robot Learning Benchmark & Learning Environment. arXiv:1909.12271, 2019.

[21] Danijar Hafner, et al. Dream to Control: Learning Behaviors by Latent Imagination. arXiv:1912.01603, 2019.

[22] Kaplan J, et al. Scaling Laws for Neural Language Models. arXiv:2001.08361, 2020.

[23] Patrick Lewis, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401, 2020.

[24] Brown TB, et al. Language Models are Few-Shot Learners. arXiv:2005.14165, 2020.

[25] M Dennis, et al. Emergent Complexity and Zero-shot Transfer via Unsupervised Environment Design. arXiv:2012.02096, 2020.

[26] A Radford, et al. Learning Transferable Visual Models From Natural Language Supervision. arXiv:2103.00020, 2021.

[27] E J. Hu, et al. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685, 2021.

[28] C Li, et al. iGibson 2.0: Object-Centric Simulation for Robot Learning of Everyday Household Tasks. arXiv:2108.03272, 2021.

[29] A Mandlekar, et al. What Matters in Learning from Offline Human Demonstrations for Robot Manipulation. arXiv:2108.03298, 2021

[30] Bommasani R, et al. On the Opportunities and Risks of Foundation Models. arXiv:2108.07258, 2021.

[31] S Ramos, et al. RLDS: an Ecosystem to Generate, Share and Use Datasets in Reinforcement Learning. arXiv:2111.02767, 2021.

[32] O Mees, et al. CALVIN: A Benchmark for Language-Conditioned Policy Learning for Long-Horizon Robot Manipulation Tasks. arXiv:2112.03227, 2021

[33] Wei J, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903, 2022.

[34] L Ouyang, et al. Training language models to follow instructions with human feedback. arXiv:2203.02155, 2022.

[35] Wang X, et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv:2203.11171, 2022.

[36] E Zelikman, et al. STaR: Bootstrapping Reasoning With Reasoning. arXiv:2203.14465, 2022.

[37] Hoffmann J, et al. Training Compute-Optimal Large Language Models. arXiv:2203.15556, 2022.

[38] M Ahn, et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. arXiv:2204.01691, 2022.

[39] J-B Alayrac, et al. Flamingo: a Visual Language Model for Few-Shot Learning. arXiv:2204.14198, 2022.

[40] S Reed, et al. A Generalist Agent. arXiv:2205.06175, 2022.

[41] M Deitke, et al. ProcTHOR: Large-Scale Embodied AI Using Procedural Generation. arXiv:2206.06994, 2022.

[42] P Wu, et al. DayDreamer: World Models for Physical Robot Learning. arXiv:2206.14176, 2022.

[43] W Huang, et al. Inner Monologue: Embodied Reasoning through Planning with Language Models. arXiv:2207.05608, 2022.

[44] M Shridhar, et al. Perceiver-Actor: A Multi-Task Transformer for Robotic Manipulation. arXiv:2209.05451, 2022.

[45] Liang J, et al. Code as Policies Language Model Programs for Embodied Control. arXiv:2209.07753, 2022.

[46] Y Jiang, et al. VIMA: General Robot Manipulation with Multimodal Prompts. arXiv:2210.03094, 2022.

[47] Yao S, et al. ReAct Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629, 2022.

[48] A Brohan, et al. RT-1: Robotics Transformer for Real-World Control at Scale. arXiv:2212.06817, 2022.

[49] Y Bai, et al. Constitutional AI: Harmlessness from AI Feedback. arXiv:2212.08073, 2022.

[50] Y Wang, et al. Self-Instruct: Aligning Language Models with Self-Generated Instructions. arXiv:2212.10560, 2022.

[51] D Hafner, et al. Mastering Diverse Domains through World Models. arXiv:2301.04104, 2023.

[52] J Li, et al. BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. arXiv:2301.12597, 2023.

[53] J Gu, et al. ManiSkill2: A Unified Benchmark for Generalizable Manipulation Skills. arXiv:2302.04659, 2023.

[54] T Schick, et al. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761, 2023.

[55] Driess D, et al. PaLM-E An Embodied Multimodal Language Model. arXiv:2303.03378, 2023.

[56] C Chi, et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. arXiv:2303.04137, 2023.

[57] N Shinn, et al. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366, 2023.

[58] A Madaan, et al. Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651, 2023.

[59] Zhao WX, et al. A Survey of Large Language Models. arXiv:2303.18223, 2023; revised 2026.

[60] Park JS, et al. Generative Agents Interactive Simulacra of Human Behavior. arXiv:2304.03442, 2023.

[61] H Liu, et al. Visual Instruction Tuning. arXiv:2304.08485, 2023.

[62] T Z. Zhao, et al. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. arXiv:2304.13705, 2023.

[63] Yao S, et al. Tree of Thoughts Deliberate Problem Solving with Large Language Models. arXiv:2305.10601, 2023.

[64] T Dettmers, et al. QLoRA: Efficient Finetuning of Quantized LLMs. arXiv:2305.14314, 2023.

[65] G Wang, et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291, 2023.

[66] Rafailov R, et al. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. arXiv:2305.18290, 2023.

[67] Lightman H, et al. Let's Verify Step by Step. arXiv:2305.20050, 2023.

[68] K Bousmalis, et al. RoboCat: A Self-Improving Generalist Agent for Robotic Manipulation. arXiv:2306.11706, 2023.

[69] A Brohan, et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. arXiv:2307.15818, 2023.

[70] Y Qin, et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. arXiv:2307.16789, 2023.

[71] Qi Wu, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155, 2023.

[72] Wang L, et al. A Survey on Large Language Model based Autonomous Agents. arXiv:2308.11432, 2023; revised 2025.

[73] H Walke, et al. BridgeData V2: A Dataset for Robot Learning at Scale. arXiv:2308.12952, 2023.

[74] C Yang, et al. Large Language Models as Optimizers. arXiv:2309.03409, 2023.

[75] Xi Z, et al. The Rise and Potential of Large Language Model Based Agents A Survey. arXiv:2309.07864, 2023.

[76] Li C, et al. Multimodal Foundation Models From Specialists to General-Purpose Assistants. arXiv:2309.10020, 2023.

[77] O Khattab, et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. arXiv:2310.03714, 2023.

[78] C Packer, et al. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023.

[79] Open X-Embodiment Collaboration, et al. Open X-Embodiment: Robotic Learning Datasets and RT-X Models. arXiv:2310.08864, 2023.

[80] A Asai, et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. arXiv:2310.11511, 2023.

[81] Y J Ma, et al. Eureka: Human-Level Reward Design via Coding Large Language Models. arXiv:2310.12931, 2023.

[82] N Hansen, et al. TD-MPC2: Scalable, Robust World Models for Continuous Control. arXiv:2310.16828, 2023.

[83] Z Zhang, Y Yao, A Zhang, et al. Igniting Language Intelligence: The Hitchhiker's Guide From Chain-of-Thought Reasoning to Language Agents. arXiv:2311.11797, 2023.

[84] W Yuan, et al. Self-Rewarding Language Models. arXiv:2401.10020, 2024.

[85] T Guo, X Chen, Y Wang, et al. Large Language Model based Multi-Agents: A Survey of Progress and Challenges. arXiv:2402.01680, 2024.

[86] Shao Z, et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300, 2024.

[87] Y Huang. Levels of AI Agents: from Rules to Large Language Models. arXiv:2405.06643, 2024.

[88] A Khazatsky, et al. DROID: A Large-Scale In-The-Wild Robot Manipulation Dataset. arXiv:2403.12945, 2024.

[89] Z Zhou, X Ning, K Hong, et al. A Survey on Efficient Inference for Large Language Models. arXiv:2404.14294, 2024.

[90] J Yang, et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793, 2024.

[91] Y Jin, J Li, Y Liu, et al. Efficient Multimodal Large Language Models: A Survey. arXiv:2405.10739, 2024.

[92] Octo Model Team, et al. Octo: An Open-Source Generalist Robot Policy. arXiv:2405.12213, 2024.

[93] Y Ma, Z Song, Y Zhuang, et al. A Survey on Vision-Language-Action Models for Embodied AI. arXiv:2405.14093, 2024.

[94] X Li. A Review of Prominent Paradigms for LLM-Based Agents: Tool Use (Including RAG), Planning, and Feedback Learning. arXiv:2406.05804, 2024.

[95] M Yuksekgonul, et al. TextGrad: Automatic "Differentiation" via Text. arXiv:2406.07496, 2024.

[96] M J Kim, et al. OpenVLA: An Open-Source Vision-Language-Action Model. arXiv:2406.09246, 2024.

[97] Y Liu, W Chen, Y Bai, et al. Aligning Cyber Space with Physical World: A Comprehensive Survey on Embodied AI. arXiv:2407.06886, 2024.

[98] Y Zhang, Z Ma, J Li, et al. Vision-and-Language Navigation Today and Tomorrow: A Survey in the Era of Foundation Models. arXiv:2407.07035, 2024.

[99] F He, T Zhu, D Ye, et al. The Emerged Security and Privacy of LLM Agent: A Survey with Case Studies. arXiv:2407.19354, 2024.

[100] Snell C, Lee J, Xu K, Kumar A. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. arXiv:2408.03314, 2024.

[101] S Hu, et al. Automated Design of Agentic Systems. arXiv:2408.08435, 2024.

[102] J Li, J Xu, S Huang, et al. Large Language Model Inference Acceleration: A Comprehensive Hardware Perspective. arXiv:2410.04466, 2024.

[103] K Black, et al. π_0: A Vision-Language-Action Flow Model for General Robot Control. arXiv:2410.24164, 2024.

[104] J Ding, Y Zhang, Y Shang, et al. Understanding World or Predicting Future? A Comprehensive Survey of World Models. arXiv:2411.14499, 2024.

[105] H Li, Y Li, A Tian, et al. A Survey on Large Language Model Acceleration based on KV Cache Management. arXiv:2412.19442, 2024.

[106] Z Li, X Wu, H Du, et al. A Survey of State of the Art Large Vision Language Models: Alignment, Benchmark, Evaluations and Challenges. arXiv:2501.02189, 2025.

[107] DeepSeek-AI, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948, 2025.

[108] Z Feng, R Xue, L Yuan, et al. Multi-agent Embodied AI: Advances and Future Directions. arXiv:2505.05108, 2025.

[109] J Zhang, et al. Darwin Godel Machine: Open-Ended Evolution of Self-Improving Agents. arXiv:2505.22954, 2025.

[110] L A Agrawal, et al. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. arXiv:2507.19457, 2025.

[111] H Gao, J Geng, W Hua, et al. A Survey of Self-Evolving Agents: What, When, How, and Where to Evolve on the Path to Artificial Super Intelligence. arXiv:2507.21046, 2025.

[112] S K S Ghasemipour, et al. Self-Improving Embodied Foundation Models. arXiv:2509.15155, 2025.

[113] T Feng, X Wang, Y Jiang, et al. Embodied AI: From LLMs to World Models. arXiv:2509.20021, 2025.

[114] X Li, X He, L Zhang, et al. A Comprehensive Survey on World Models for Embodied AI. arXiv:2510.16732, 2025.

[115] Z Mei, T Yin, O Shorinwa, et al. Robotic Video World Models: A Survey of Applications, Research Challenges, Future Directions. arXiv:2601.07823, 2026.

[116] T Feng, X Wang, W Zhu. Self-evolving Embodied AI. arXiv:2602.04411, 2026.

[117] S Ye, et al. World Action Models are Zero-shot Policies. arXiv:2602.15922, 2026.

[118] J Zhang, et al. Hyperagents. arXiv:2603.19461, 2026.

[119] He C, Zhou X, Wang D, et al. Harness Engineering for Language Agents: The Harness Layer as Control, Agency, and Runtime. Preprints.org, 2026; v2. https://www.preprints.org/manuscript/202603.1756

[120] Meng Q, Wang Y, Chen L, et al. Agent Harness for Large Language Model Agents: A Survey. Preprints.org, 2026; v3. https://www.preprints.org/manuscript/202604.0428

[121] B Hou, G Li, J Jia, et al. World Model for Robot Learning: A Comprehensive Survey. arXiv:2605.00080, 2026.

[122] S Wang, J Shi, Z Fu, et al. World Action Models: The Next Frontier in Embodied AI. arXiv:2605.12090, 2026.

[123] Ning X, et al. Code as Agent Harness. arXiv:2605.18747, 2026.

[124] Z Feng, Q Li, H Liang, et al. From Human Videos to Robot Manipulation: A Survey on Scalable Vision-Language-Action Learning with Human-Centric Data. arXiv:2606.00054, 2026.

[125] D Kim, D Park, S Lee, et al. Safe Embodied AI for Long-horizon Tasks: A Cross-layer Analysis of Robotic Manipulation. arXiv:2606.05660, 2026.

[126] J Guo, Z Hao, C Wang, et al. From Question Answering to Task Completion: A Survey on Agent System and Harness Design. arXiv:2606.20683, 2026.

[127] Q Shen, S Zhang, Y Liao, et al. World Action Models: A Survey. arXiv:2606.20781, 2026.

[128] Jiang C, Zhong J, Fu Y, et al. Self-Improving Agents in the Era of Experience: A Survey of Self- to Meta-Evolution. OpenReview preprint, 2026. https://frontisai.github.io/Awesome-Self-Improving-Agents/

[129] M Chen, L Wang, B Qu. Recursive Self-Improvement in AI: From Bounded Self-Refinement to Autonomous Research Loops. arXiv:2607.07663, 2026.

[130] Z Ren, Y Chen, D Guo, et al. Self-Improvements in Modern Agentic Systems: A Survey. arXiv:2607.13104, 2026.

[131] Y Ye, et al. Data Pyramid for Embodied Manipulation: A Survey. arXiv:2607.24744, 2026.

[132] Nguyen LX, Raha AD, Le HQ, and et al. Self-Evolving AI for Humanoids: Mechanisms, Safety, and Evaluation of Post-Deployment Self-Improvement. arXiv:2609.13236, 2026.

[133] Y Duan, et al. The Last AI Built by Humans: Toward Genuine Recursive Self-Improvement. arXiv:2609.11873v3, 2026.

[134] Z Lu, H Zhai, G Wang, et al. World-Action Models for Robot Learning and Control: A Survey. arXiv:2609.16074, 2026.

[135] M Piccinini, L Schulze, A Plebe, et al. Embedding Physics Priors in Robot Learning: A Survey. arXiv:2609.22319, 2026.

[136] H Liu, T Ye, S Gao, et al. SoL-Pi: Recursively Scaling Auto-Research Loops for Efficient Agent Harness. arXiv:2609.20519, 2026.

[137] P Xia, et al. RRSI: Regularized Recursive Self-Improvement of Agent Harnesses. arXiv:2609.24972, 2026.

[138] D Srikanth, et al. Recursive self-improvement of AI research agents. arXiv:2609.26457, 2026.

[139] Z Shen, et al. EmbodiedSWE: Coding Agents for Long Horizon Dexterous Robotics. arXiv:2609.27308, 2026.

[140] K Wang, et al. RegenHarness: A Robot Agent Harness with Evidence-Gated Recursive Self-Improvement. arXiv:2609.27612, 2026.

[141] J Wang, et al. What Stops Recursive Self-Improvement in Robotics? Lessons from 123 Rounds of Agentic Skill Discovery. arXiv:2609.31760, 2026.

[142] Huang Y, Chen Y, Yang Z, Ding G. Embodied Agentic Harnesses for Physical Intelligence A Survey. engrXiv preprint, 24 September 2026. https://engrxiv.org/preprint/view/8297

[143] C Gong, J Bi, D Yao, et al. Evolutionary Safety of Recursive Self-Improving AI: Taxonomy, Risk Discovery, and Evaluation. arXiv:2609.31186, 2026.

[144] J Liang, et al. RoboFoundry: System-as-Policy Evolution for Self-Learning Embodied Agents. arXiv:2609.32862, 2026.

[145] H Gao, J Zhou, Z Zheng, et al. Self-Evolving Coding Agents: From Digital Programs to Physical-World Intelligence. arXiv:2609.35432, 2026.

[146] H Huang, Z Li, J Guo, et al. In-Context Learning for Robots: Methods and Applications. arXiv:2609.36012, 2026.

[147] S Bao, et al. RoboHarn-Evo: Evolving Hierarchical Physical Knowledge for Self-Improving Robotic Manipulation. arXiv:2609.37583, 2026.

[148] B Li, Siqi Song, Yizhuo Wu, et al. MotorMind: Scaffolding General Vision Language Models for Zero-Shot Robot Manipulation. arXiv:2609.38078, 2026.

[149] H Lang, et al. EmbodiRSI: Recursive Self-Improvement for Data-Efficient Robot Adaptation. arXiv:2609.38905, 2026.

[150] Haoyuan Deng, et al. DynaHarness: A Dynamic Physical Harness for Self-Evolving Robot Agents. arXiv:2609.40306, 2026.

[151] Wu H, Xu F, Zhang F, et al. Recursive Self-Improvement in AI: A Survey. Preprints.org, 30 September 2026. https://www.preprints.org/manuscript/202609.2680

[152] C Liu, et al. MobiAgent: Dual-Loop Recursive Policy Self-Improvement for Long-Horizon Mobile Manipulation. arXiv:2610.03476, 2026.

[153] Y Na, et al. EvoMem-VLA: State-Evolution Memory for Long-Horizon Robot Manipulation. arXiv:2610.05418, 2026.

[154] Y Qin, et al. EmbodiedSmith: Scaling Embodied Data through Recursive Self-Improvement Flywheel in Simulation. arXiv:2610.07969, 2026.

[155] W Tian, Z Zhang, Z Liu, et al. PhysEvo: Astra Can Act, Let It. arXiv:2610.08995, 2026.

[156] R Xu, et al. Harness^2: Recursive Agent Harnessing for an Open World. arXiv:2610.14224, 2026.

[157] Z Feng, et al. EnactOpt: Diagnosis-Guided Harness Evolution for Embodied Agents. arXiv:2610.14977, 2026.

[158] X Liu, Yutong Jiang, Jian Su, et al. Harnessing VLM in Embodied RL: From Reward Regressor to Success Detector via Exploration-Augmented Key-Observation Gating. arXiv:2610.15401, 2026.

[159] X Wen, et al. D-World: An Explicit World-Language-Action Model with Recursive Self-Improvement under Dual Control. arXiv:2610.17114, 2026.

[160] H Zhou, et al. Who Improves the Robot? Toward Physical Recursive Self-Improvement. arXiv:2610.18842, 2026.

[161] Z Lin, et al. VideoHarness-SRSI: Structured Recursive Self-Improvement for Video Understanding Harness. arXiv:2610.19424, 2026.

[162] C Qin, et al. Towards Agentic Robots: Model, Data, Environment, and Harness. arXiv:2610.20442, 2026.

[163] P Wang, et al. AdaRSI: On-Demand Recursive Self-Improvement for Embodied Agents. arXiv:2610.20474, 2026.

[164] Z Wang, et al. CarryOn: A Dual-Loop Harness for Personalized and Reliable Robot Assistance. arXiv:2610.21453, 2026.

[165] Li J, Xiao X, Zhang Y, et al. Agent Harness Engineering: A Survey. OpenReview preprint, 2026. https://picrew.github.io/LLM-Harness/

[166] RoboCasa365. Official project and data resources. https://robocasa.ai/docs/build/html/datasets/datasets_overview.html (accessed 19 October 2026).

[167] LIBERO. Official project and data resources. https://libero-project.github.io/datasets.html (accessed 19 October 2026).

[168] LIBERO-PRO. Official project and data resources. https://github.com/Zxy-MLlab/LIBERO-PRO (accessed 19 October 2026).

[169] VLABench. Official project and data resources. https://vlabench.github.io/ (accessed 19 October 2026).

[170] ALFWorld. Official project and data resources. https://github.com/alfworld/alfworld/blob/master/alfworld/data/README.md (accessed 19 October 2026).

[171] Embodied Agent Interface EAI. Official project and data resources. https://embodied-agent-eval.readthedocs.io/en/latest/modules/action_sequencing.html (accessed 19 October 2026).

[172] EmbodiedBench. Official project and data resources. https://embodiedbench.github.io/ (accessed 19 October 2026).

[173] BEHAVIOR-1K. Official project and data resources. https://behavior.stanford.edu/getting_started/important_concepts.html (accessed 19 October 2026).

[174] RMBench. Official project and data resources. https://github.com/RoboTwin-Platform/RMBench (accessed 19 October 2026).

[175] RoboMME. Official project and data resources. https://github.com/RoboMME/robomme_benchmark/blob/main/readme.md (accessed 19 October 2026).

[176] EWMBench. Official project and data resources. https://github.com/AgibotTech/EWMBench (accessed 19 October 2026).

[177] WorldArena. Official project and data resources. https://github.com/tsinghua-fib-lab/WorldArena (accessed 19 October 2026).